

数字系统设计

Digital System Design

-----微控制器之

-微码控制器与流水结构

王维东

Weidong Wang

浙江大学信息与电子工程学院

College of Information Science & Electronic Engineering

信息与通信网络工程研究所

Zhejiang University

Spring 2026 – Lec. #09

任课教师

- 王维东
- 浙江大学信息与电子工程学院, 信电楼306
- 邮箱: wdwang@zju.edu.cn
- College of Information Science & Electronic Engineering
- Zhejiang University, Hangzhou, 310027
- Tel: 86-571-87953170 (O)
- Mobile: 13605812196

TA:

钟睿, 13687791509; 22331166@zju.edu.cn;

Office Hours: 玉泉信电楼 308室 (可以钉钉、微信或邮件联系) .

Prerequisites预修课程

- 电子电路基础
- 电子线路
- C语言
- How to learn this Course?
 - Not only listening, thinking and waiting
 - But Exercise, Simulation, Practice!

课程简介

- 课程代码: **671C0050**
- 参考书
 - 阎石,《数字电子技术基础》,第6版,高等教育出版社, 2016.
 - 王金明著,《数字系统设计与Verilog HDL》, 电子工业出版社, 第8版
 - 《数字系统设计-补充讲义》, 浙江大学自编讲义
 - Stanford 大学 108A课程notes.
 - R.H.Katz, G.Borriello, *Contemporary Logic Design*, second edition, 电子工业出版社, 2005.
 - M.M.Mano, 数字设计(第四版), 电子工业出版社, 2010.
 - » <http://www.prenhall.com/mano>

Other Course Info

- **Website:**

- <http://mypage.zju.edu.cn/wdwd/教学工作/>
- **学在浙大: 数字系统设计**
- Check frequently

- **答疑**

- 玉泉信电楼308室/周一周四下午2:30-5:30
- 上课课间、课后均可
- Email, 钉钉/微信群/数字系统设计, 短信均可

Grading (考核)

Final grades will be computed approximately as follows:

- 平时（含课程作业、期中考试、小测验、Project、出勤等） **40%**

- 期中考试 **10%**
- Homework Sets **15% (12~20)%**
- Project **5% (5~7)%**
- Class Room Check+小测验等 **10% (7~10)%**
- 作业每周一上交, **截止期为课后一周内**有效。

- **Project**

- Project-1必做 (1 members)
- Project-2可选 (成绩加分1%~5%, 但不超过平时成绩范围)

- Finial Exam 期末闭卷考试 - **60%**

授课时间和地点:

- 2025年春夏学期,
 - 周一上午, 第1~2节(8:00-9:35)
 - 星期四上午, 第6~7节(13:25-13:00)
- 地点: 紫金港西2-415(多)
- <http://mypage.zju.edu.cn/wdwd/教学工作/>
- 学在浙大/数字系统设计
- 钉钉/微信/数字系统设计-答疑

课程结构

- 数字理论知识(必备)
 - 数字系统和编码、逻辑代数、门电路
- 数字电路分析与设计
 - 组合逻辑电路
 - 触发器、半导体存贮器、可编程器件
 - 时序逻辑电路
- 脉冲电路与接口
- 控制器与数字系统
 - 状态机
 - 控制器
 - 微码控制器
 - 测试和验证
- 微处理器简介
 - 指令集
 - **4位CPU**介绍

控制单元与数据通道（复习）

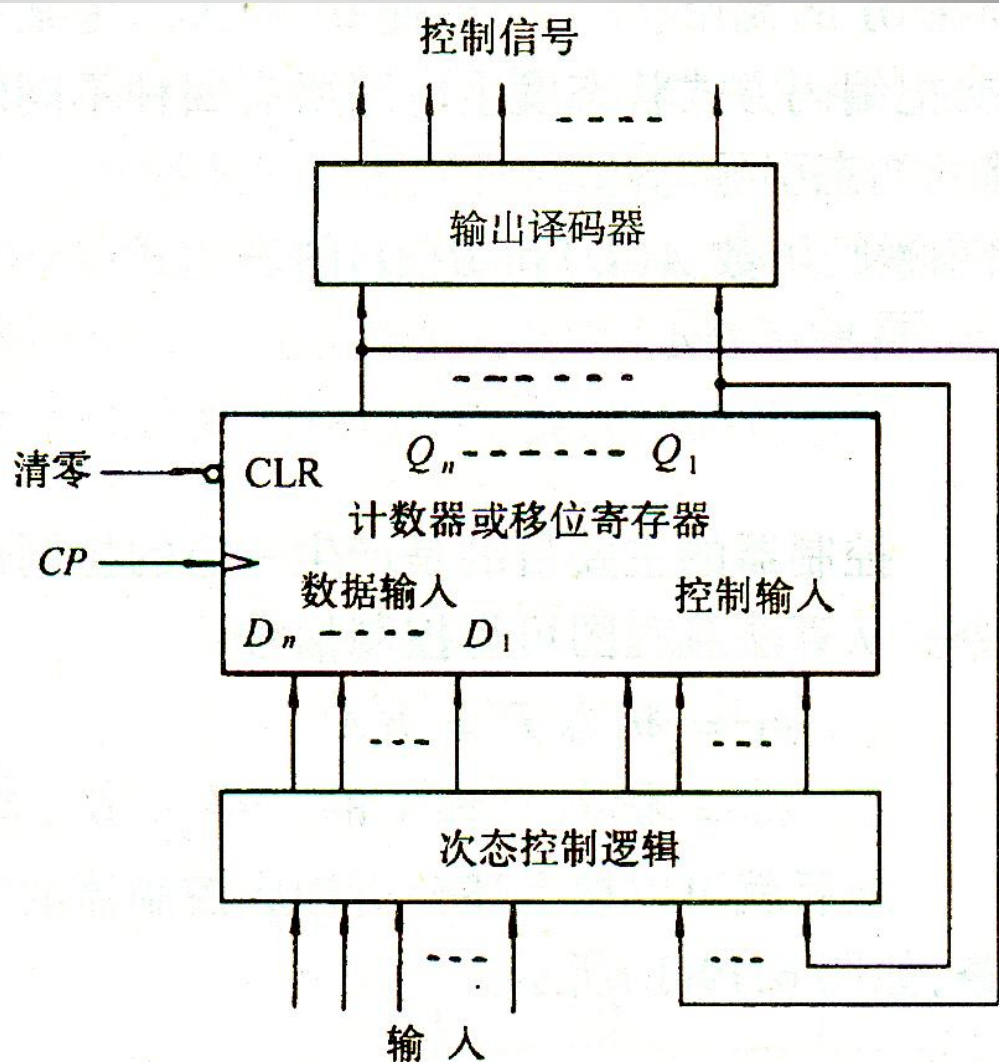
- **ASM图给出了设计数字系统（控制电路和数据路径）需要的所有信息。**
 - 控制电路硬件与数据路径的硬件划分。
- **数据路径包含了与其操作和逻辑所关联的硬件，这些逻辑用于产生提供给控制电路的状态信号。**
 - 数据路径的设计要求在ASM图的状态框和条件框中给出，由在数据路径上标注操作来确定。
- **控制单元包含了所有数据路径中操作控制信号逻辑。**
 - 控制逻辑有判决框和所需状态转移决定。

控制逻辑（复习）

- 数字系统的设计过程可分成两个部分：
 - 数据路径中的寄存器传输设计和控制单元中的控制逻辑设计。
 - 控制逻辑是一个有限状态机，其米里型和摩尔型输出控制着数据路径中的操作。
 - 控制单元的输入是外部输入，内部状态信号从数据路径反馈到控制电路。
- 控制电路是时序电路，可以采用前面概括出的时序逻辑设计步骤进行设计。
 - 现在提出的方法是对经典时序电路设计方法的补充。
- 设计方法
 - 采用序列寄存器-计数器和译码器
 - 采用数据选择器来设计
 - 采用一位热位设计（每个状态一个触发器）

1-计数型控制器

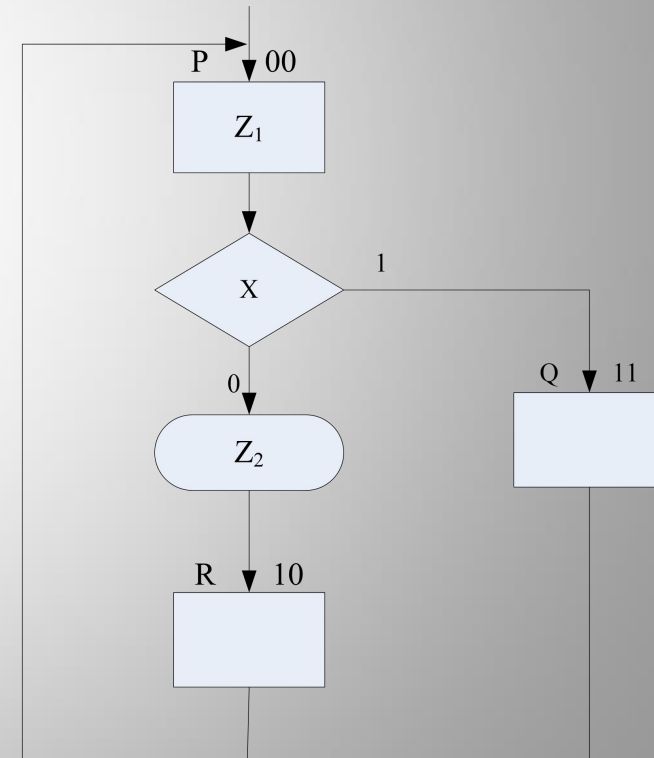
- 将所要求的**控制状态**按一定原则进行**编码分配**，就可设计出一种**状态计数**型的控制器。
- 图中计数器含有多个触发器，触发器的状态作为状态变量以**二进制编码**的形式赋于ASM流图中的每一个状态框，而条件输出框不予赋值。
- 使用寄存器产生控制状态，使用**译码器**产生与每个状态对应的**输出信号**。若使用一位热位编码，就不需要使用译码器。



2-采用数据选择器来设计控制器

- 三级电路结构：
 - 决定寄存器次态的数据选择器组成
 - 保存现态的寄存器
 - 译码器，产生每个控制状态对应的输出
- 设计一个多路选择器型控制器，完成左图的控制算法。
- 根据算法流程图，得到状态转移表

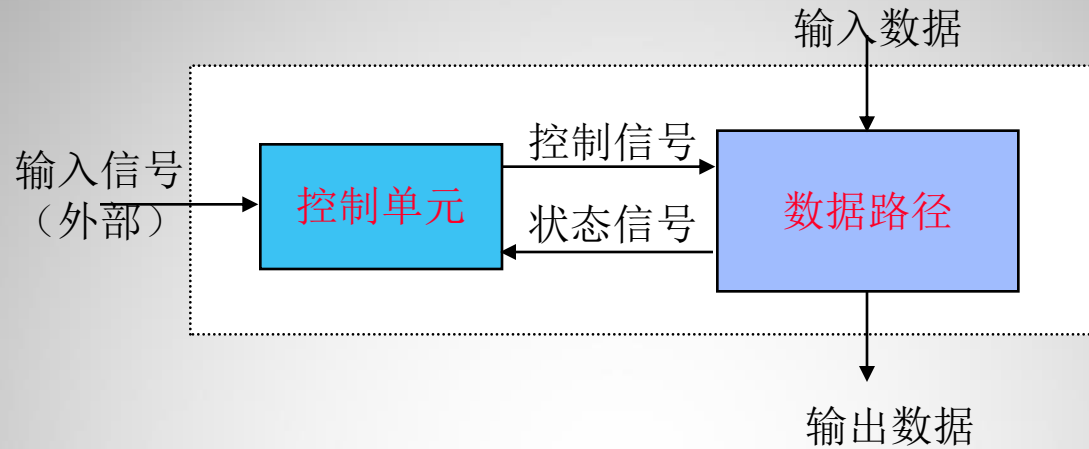
PS(现态)		NS(次态)			转换条件
编码	状态名	状态名	B	A	
0 (00)	P	R	1	0	$\overline{X}$
		Q	1	1	X
2 (10)	R	P	0	0	0
3 (11)	Q	P	0	0	0
1 (01)	–	P	0	0	0



3-定序型控制器

- 一位热位（**One-hot**）设计，产生的电路每一个状态需要一个触发器。每次只有一个触发器为1，其余触发器都为0。
- 触发器的数目代表了**状态数**，并依赖一组最新的代码实现状态转换。
- 使用一位热位的方法会在时序电路中增加很多触发器。但是一位热位设计方法的优点是**设计简单, 不需要译码器**。

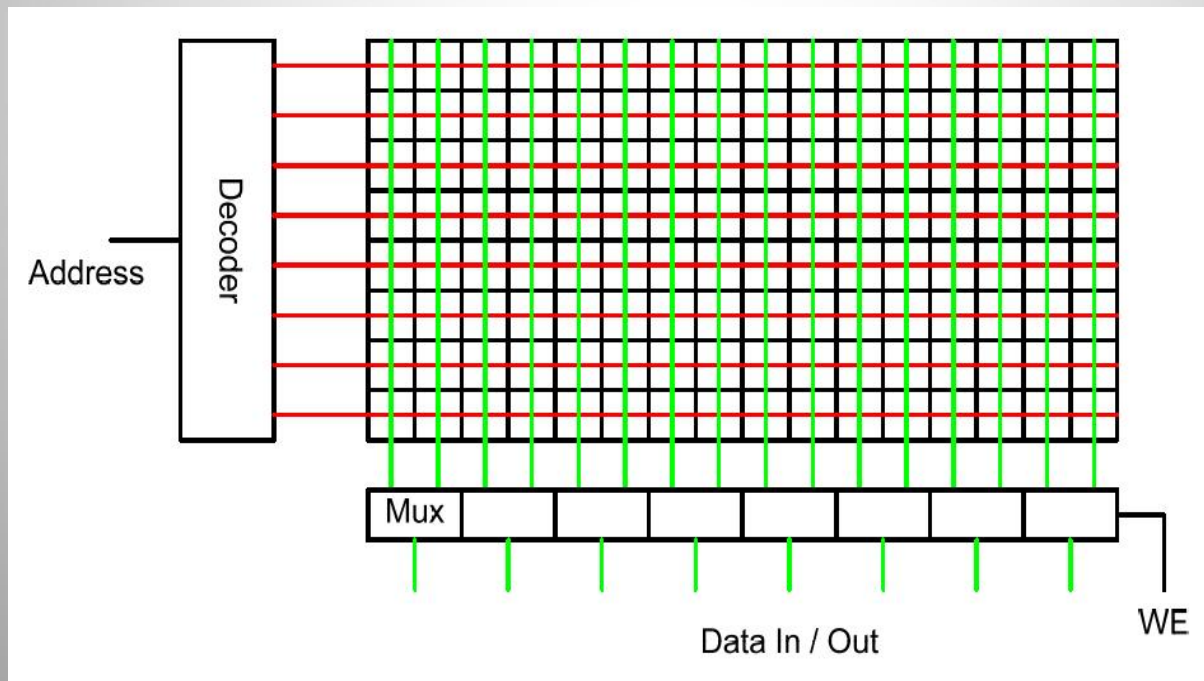
控制和数据通道的交互



- **Algorithmic state machine** and **datapath (ASMD) charts** were developed to clarify the information displayed by ASM charts and to provide an effective tool for designing a control unit for a given datapath unit.
- **An ASMD chart associate register** operations with state transitions rather than with states.
- In practice, designers **use the ASMD chart to write Verilog models** of the controllers and the datapath and then synthesize a circuit directly from the Verilog description.

Memory Structure

- **Memory structures are generally specially designed**
 - Could build them from flops or latches
 - » But they would be big, slow, and power hungry
 - So circuit designers create the basic design
 - » Create a module generator for logic designers to use

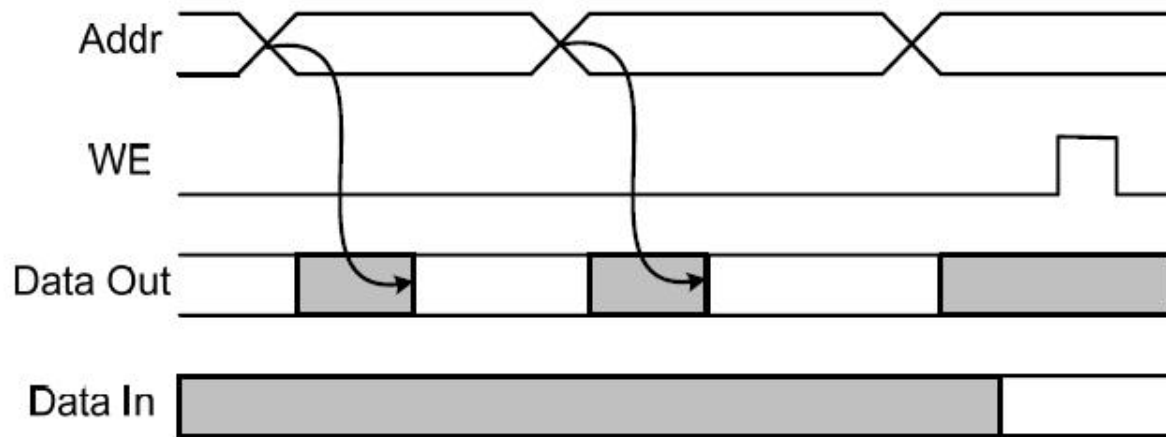


Read from/Write to Memory

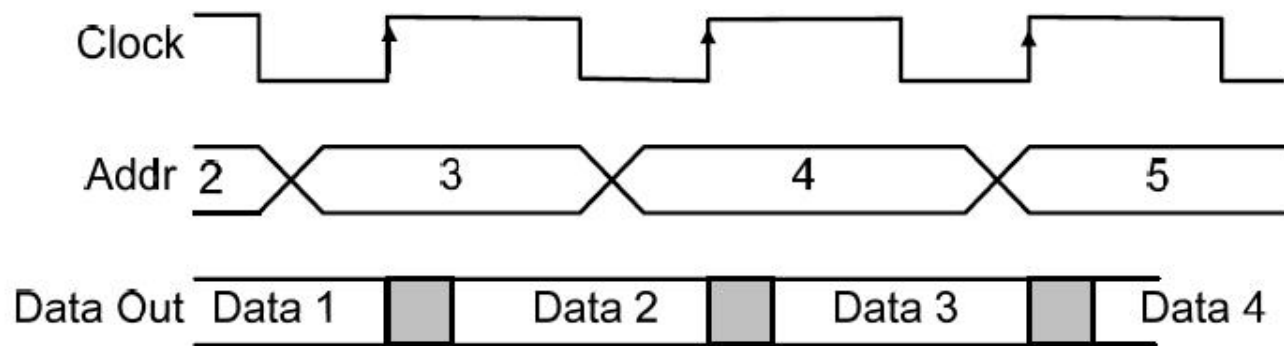
- **Interface to Memory can be:**
 - Combinational (asynchronous)
 - Clocked (synchronous)
- **Combinational memory:**
 - Read data is valid some delay after address lines settle
 - » There is no clock.
 - Writes are tricky: must supply a write pulse in the middle of your address and data valid times
- **Clocked memory (most common):**
 - Memory looks like a standard synchronous device.
 - Address and control signals are sampled on rising edge of clock, and data is valid some number of cycles later

Memory Timing

- Combinational/Asynchronous:



- Synchronous



Memories In Our Design

- **They will be combinational**

- Otherwise we can't complete an instruction in one cycle!

- **Interface is simple:**

- Inputs:

- » Address

- » DataIn

- » WriteEn (WriteEn must be a pulse)

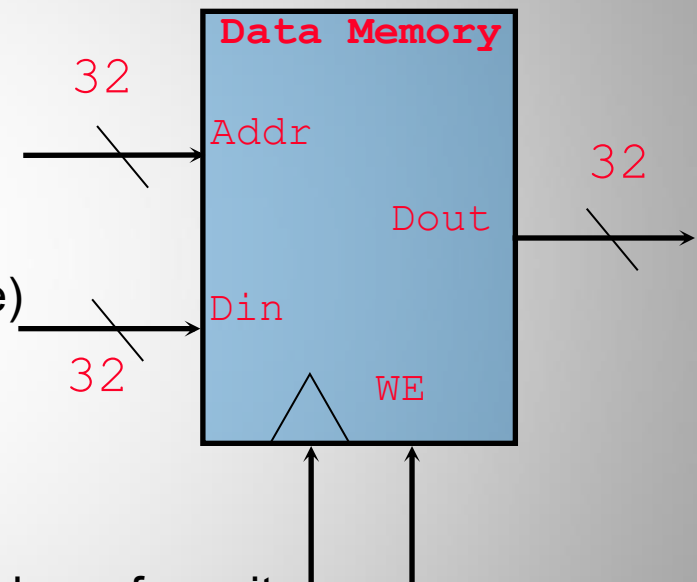
- Outputs:

- » Dataout

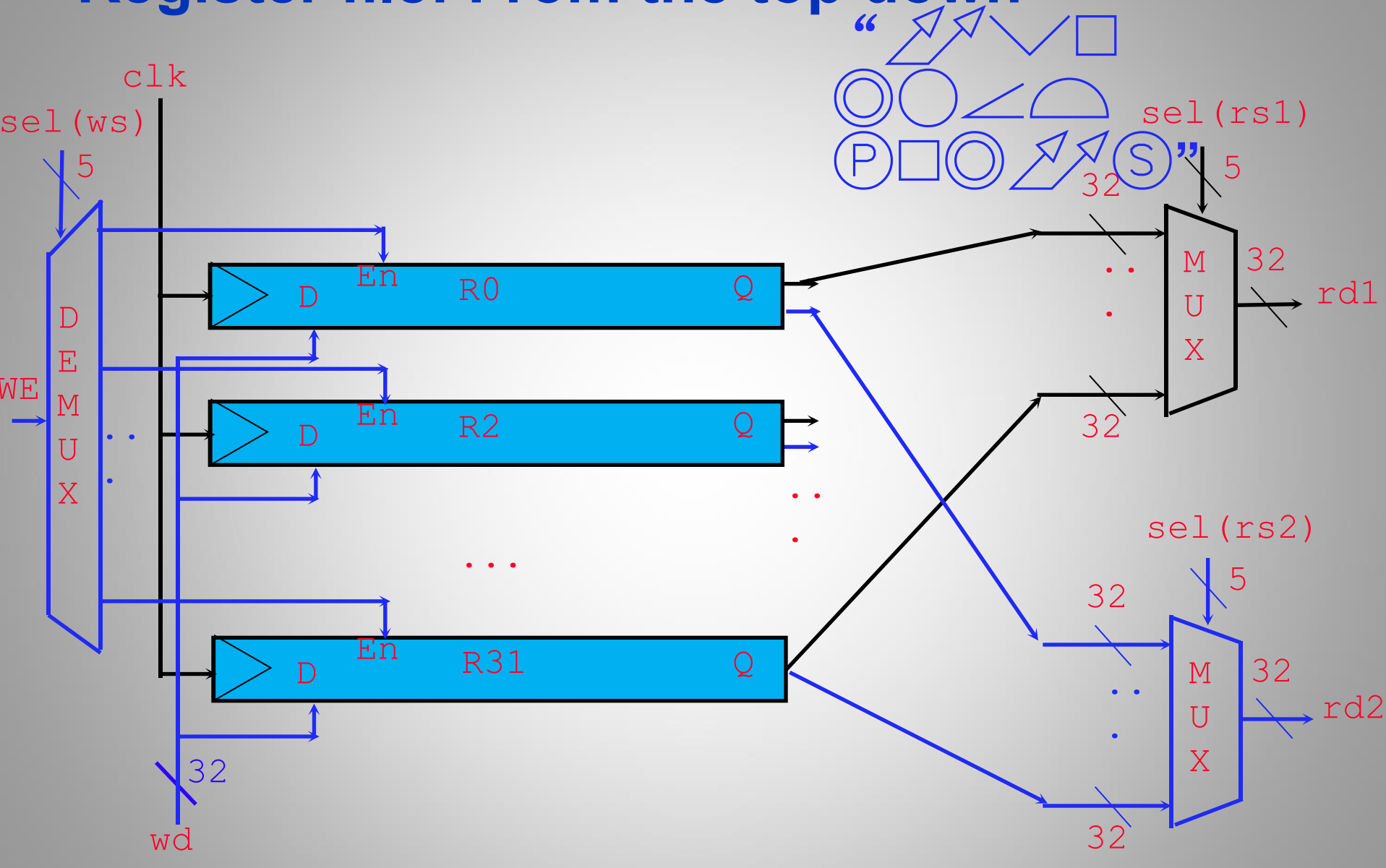
- **Register file:**

- It has three address, two for reads, and one for write

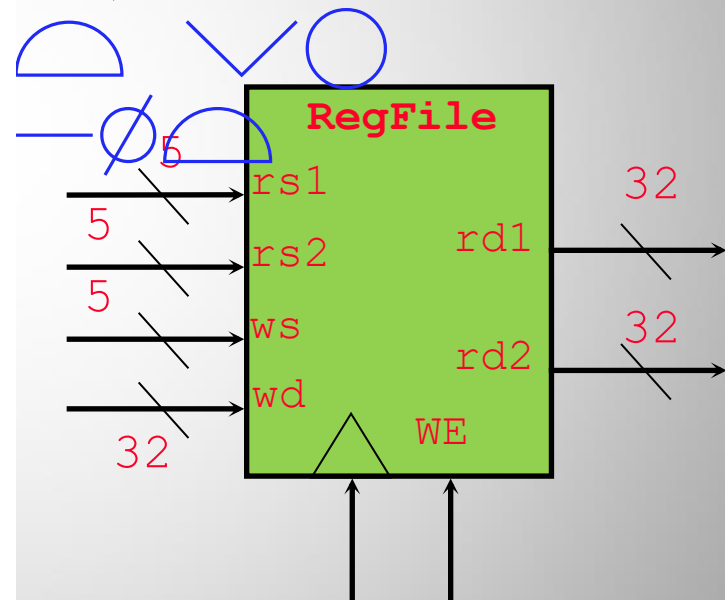
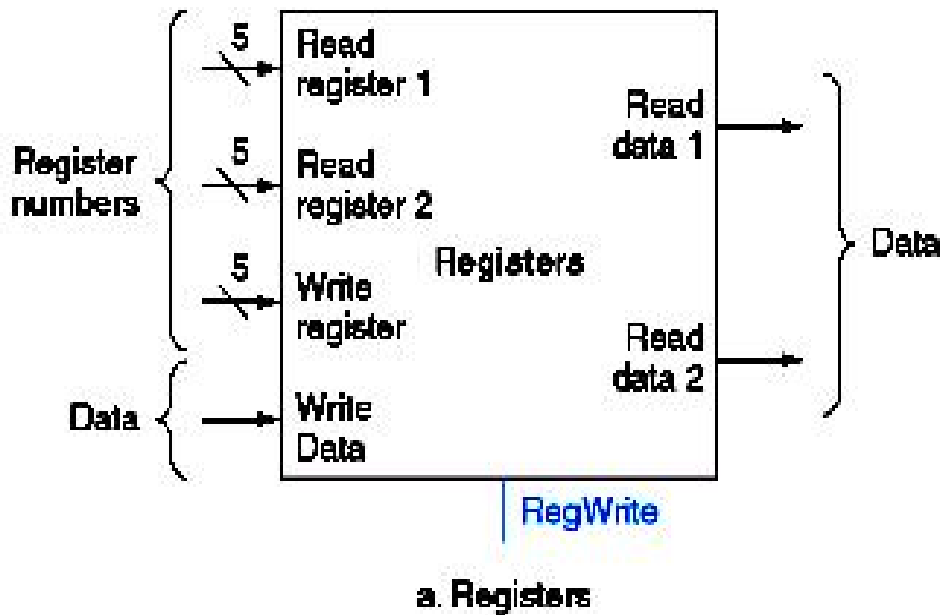
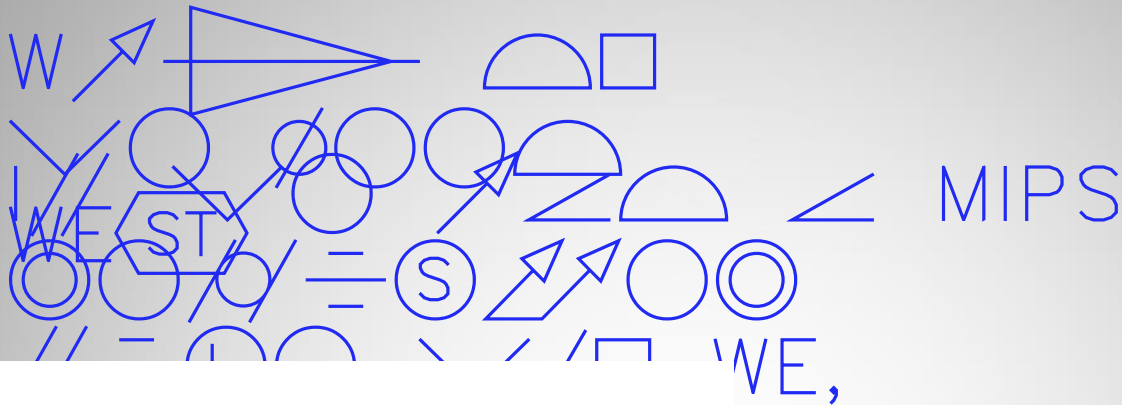
- It is called a 3-port, since it can perform 3 accesses per cycle



Register file: From the top down



Register File Schematic Symbol



讲义第8章 微控制器设计

-----微码控制器(重点)

8.4 微程序(码)控制器

- 微程序控制的基本思想

- 仿照通常的解题程序，把所有控制信号汇集一起编码，称为**微指令**，存放在一个**EPR0M**存储单元里。
- 系统运行时，一条一条地读出这些微指令，解码产生执行部件所需的各种控制信号，控制各逻辑部件执行规定的操作。

- 数据总线

- 数据流通过**BUS**总线在各执行子系统之间进行流动

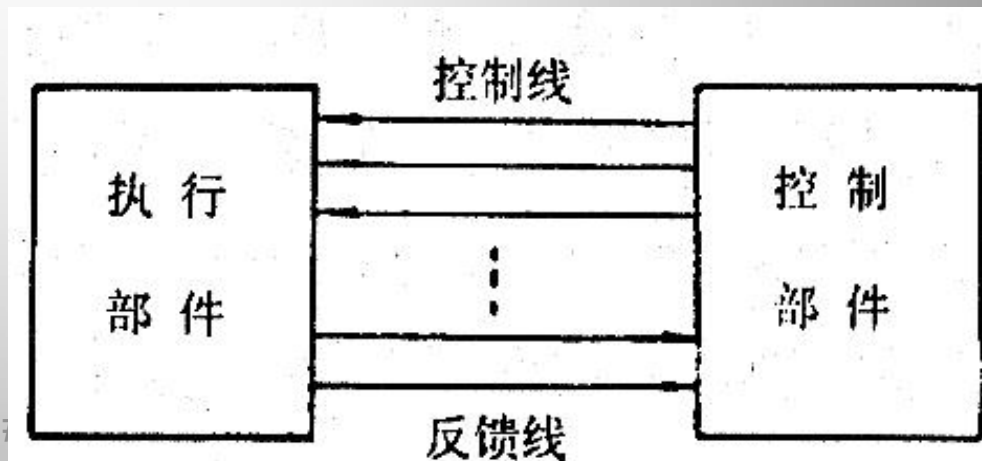
- 执行部件

- ALU、寄存器组、存储器**RAM**等

- 控制部件

- 控制器

控制部件通过控制线向执行部件发出各种控制命令，我们把这种控制命令称为**微命令**，而执行部件接受微命令所执行的操作叫作**微操作**。



基本子系统（补充）

- 基本子系统

- 是指构成数字系统时必不可少的逻辑功能部件。

- » 算术逻辑运算单元ALU、寄存器、RAM、数据总线和控制器。

- 算术逻辑运算单元ALU

- 它是数字系统中对数据进行加工处理的功能部件。

- ALU的结构

- （1）最简单的算术运算单元 -- 加法器

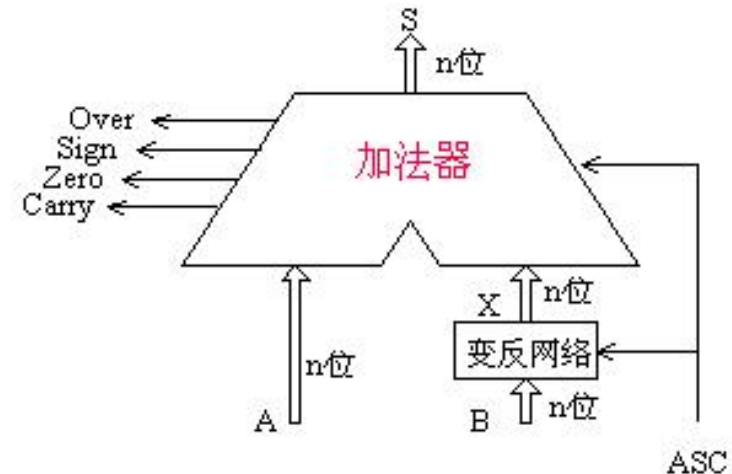
ASC为控制信号；**A**、**B**是参加运算的两个二进制数（**n**位）；**S**是求和结果信号。

控制信号**ASC=0**时，进行加法运算。**B**数经过变反网络后仍保持原数，即 $X=B$ 。运算结果为 $S=A+B$ 。

控制信号**ASC=1**时，进行减法运算。**B**数经过变反网络后

–
$$\bar{X} = \bar{B}, \quad S = A + \bar{B} + 1.$$

在这里减法是按2的补码运算规则进行的，+1是用**ASC=1**来体现。此处将减法运算转化为加法运算，简化了加法器的设计。



算术运算单元

• 最简单的算术运算单元

- 加法器左边四个输出信号是运算时提供的几个结果状态信号，分别需要一个触发器进行保存，以便在设计控制器时做为反馈信号。

- » **Over**--- 溢出标志信号，当其值为1时，表示运算结果发生溢出。
- » **Sign**--- 表示运算结果的符号（正或负）。
- » **Carry**--- 表示最高位的进位输出信号。
- » **Zero**--- 当加法器的求和结果 $S=0$ 时，标志信号 $Zero=1$ 。

• (2) 算术逻辑运算单元ALU

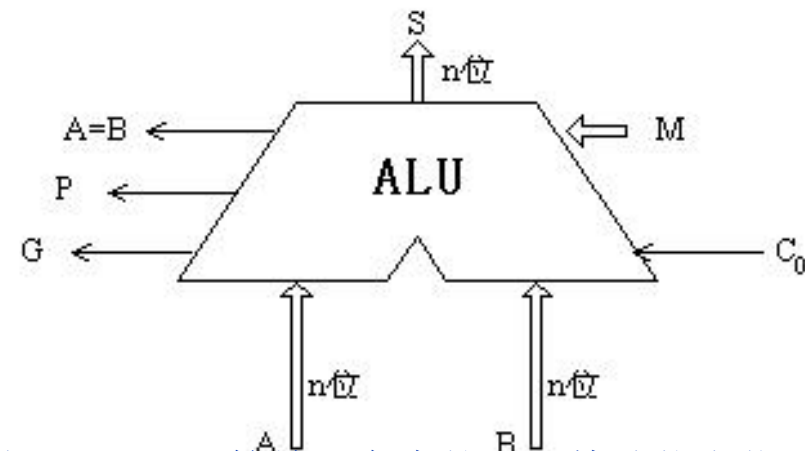
- 加法器仅提供加、减、传送、加1等很少功能，不具备逻辑运算能力。而ALU可以实现多种算术运算和逻辑运算。

M为控制器参数。根据**M**的位数多少，**ALU**可设计成具有各种运算功能。例如芯片**AM2901**中，**M**有3位，因而**ALU**的功能选择有8种。

又如芯片**74LS181**中，控制参数有**S3**、**S2**、**S1**、**S0**、**M**共5个，因而组合可以实现16种算术运算和16种逻辑运算。

其中加表示算术加，+ 表示逻辑加。

为了设计控制器，ALU运算结果产生的一些特殊状态信号需要用一些标志触发器保存起来。



寄存器堆

读出操作：读数时，控制信号**RD**有效，所以由**A**地址和**B**地址指定的两个寄存器的数据分别送出到端口**A**和端口**B**。

写入操作：先给出**B**地址，待存入的数据放到数据输入端，当控制信号**WR**有效时，数据按**B**地址指定的寄存器编号写入到该寄存器。

• 寄存器的功能

- 加法器和**ALU**均由门电路组成，它们没有记忆功能，因此运算的结果需要寄存器保存起来。而参与运算的两个数也要取自寄存器。寄存器是数字系统中必不可少的逻辑子系统。

• 寄存器的分类

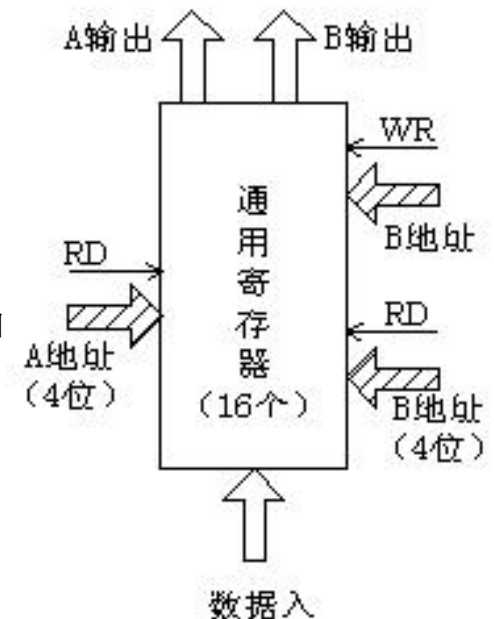
- 数字系统中使用的寄存器大体有两类：

» (1) 通用寄存器

- 用来暂存参与**ALU**运算的数据或结果。通用寄存器的数目一般是4、8、16、32个，甚至更多。

» (2) 专用寄存器

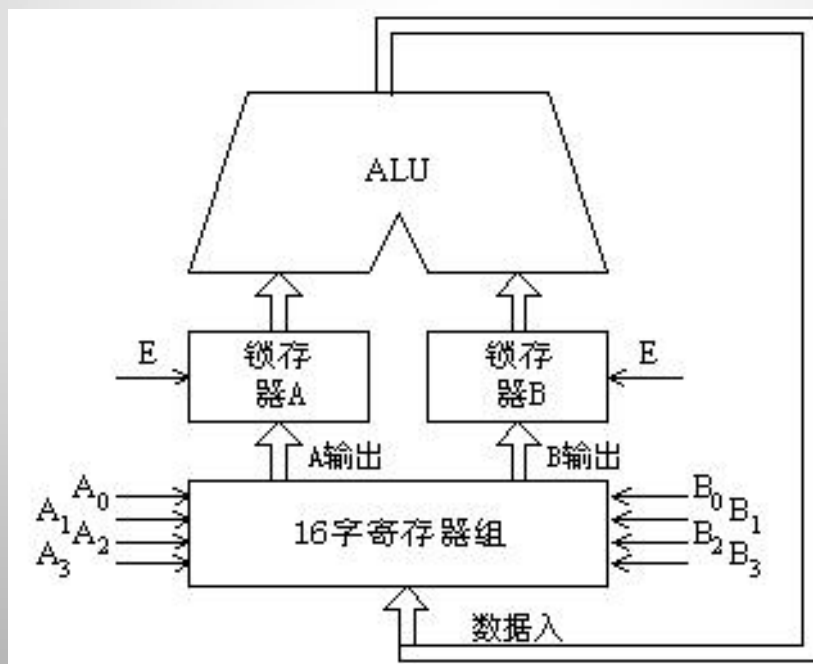
- 数字系统中的专用寄存器根据不同系统而有所不同。例如**ALU**的状态标志信号寄存器、地址寄存器、数据缓冲寄存器、指令寄存器、程序计数器等等。
- 他们用来存放地址信息和控制器的控制信息。



双端输出寄存器堆（组）的功能框图。它有一个数据输入端口，两个数据输出端口，可以同时从寄存器堆中取出**A**、**B**两个数。寄存器的选择由**A**地址或**B**地址指定。由于地址线是4位，因而有16个通用寄存器。

双端口通用寄存器堆

- 双端口通用寄存器堆的结构非常有用
- 它可以与ALU一起构成一个最简单的运算器。
- 为了时间上进行缓冲，ALU和通用寄存器堆之间加入了两个锁存器。



存储器RAM

读操作：控制信号**RD**有效，将地址译码器选中的存储单元的内容读出到数据缓冲寄存器**MDR**，然后送到数据总线上。

写操作：控制信号**WR**有效，将写入的数据由数据总线送到**MDR**寄存器，然后写入存储矩阵。

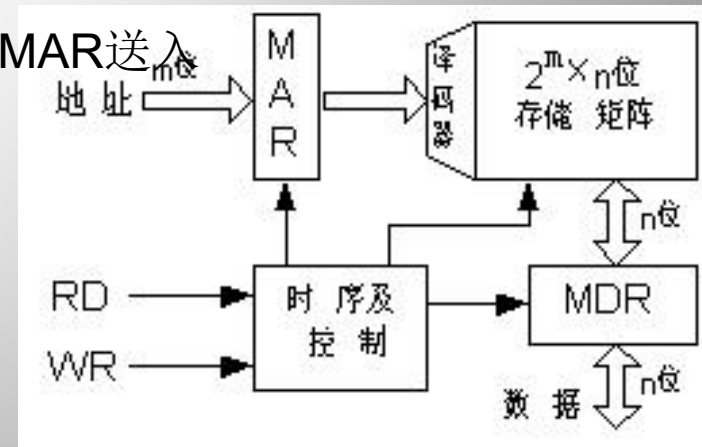
注意：存储器的读写操作是分时进行的：读时不写，写时不读。另外同**MDR**连接的 n 位数据线具有双向传送功能。

• 存储器RAM

- 一个触发器存储一个二进制位（**bit**）信息；
- 一个寄存器存储一个字（若干 **bit**）信息；
- 一个寄存器堆也只能存储有限个字的信息。当存储大量数据时，从经济和成本上考虑，只能使用随机读写的**RAM**存储器。

• RAM的逻辑结构图。

- 其中**MAR**是地址寄存器，通过地址译码器译码，可选中相应的存储单元。
- **MDR**是数据缓冲寄存器，读出的数据或写入的数据都由**MDR**暂存。存储容量为 2^m 个字（字长 n -bit）。**RD**和**WR**是RAM的读、写操作控制信号。
- 不论是读操作还是写操作，首先要通过地址线向**MAR**送入地址信息。



数据通路（补充）

• 总线结构

- 在数字系统中，总线是多个逻辑子系统的联系纽带。
- 所谓总线，就是多个信息源分时传送数据到多个目的地的传送通路。
- 如果总线的始端和终端是固定不变的，即信息只能从始端向终端传送，称为单向总线。
- 如果信息的源端与目的端是相对的，称为双向总线，它可以实现信息的双向传送

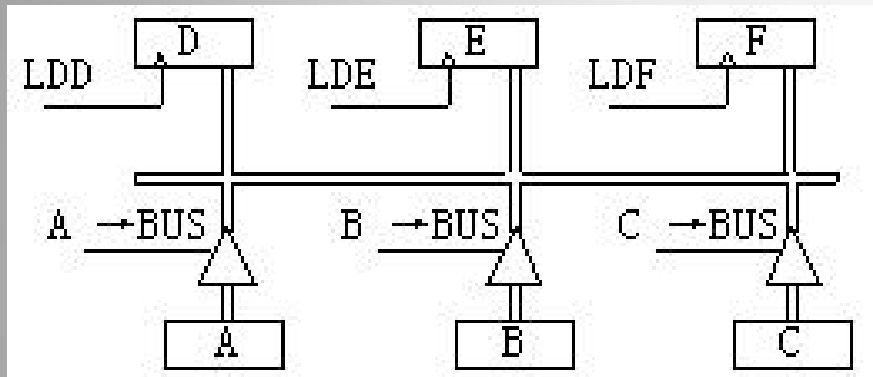


总线原理示意图。总线始端有A1、A2、A3、A4四个信息来源，经总线传输后有四个输出B1、B2、B3、B4。传送数据时，只允许一个数据进入总线。换句话说，某一时刻只能传送 A1-A4 四个信息源中的一个，这就需要在总线始端对进入总线的信息有选择地加以控制。同样，总线终端输出数据要送往何处，也需要有选择地加以控制。这个任务由控制器来完成。

总线结构

• 总线的逻辑结构

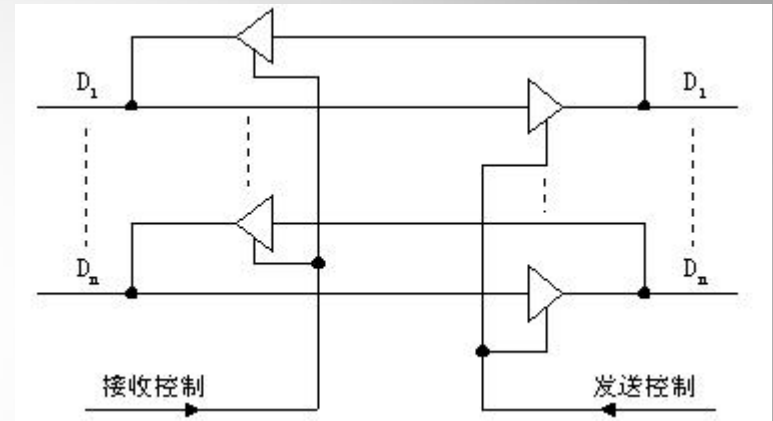
- 总线结构的逻辑实现可以实现多路选择器方式、三态门方式。
- 多路选择器方式是单向总线。三态门方式可以组成双向总线。



三态门构成的总线。发送数据的三个寄存器A、B、C通过三态门与总线BUS相连接；接收数据的寄存器D、E、F直接接在BUS上，并用寄存器的时钟端作为接收控制信号。

当三态门的使能控制信号有效时，发送寄存器的数据发送到BUS上，一旦接收寄存器的接收控制信号有效，便将数据打入到某个或几个寄存器中。

当三态门使能控制信号无效时，该三态门输出端呈高阻态，相当于该三态门与BUS断开。三态门的这种特性，保证了总线上信息的分时传送。



双向数据总线的逻辑结构图。图中只画出总线中的两位。接收控制信号与发送控制信号由控制器给出，它们分别加到三态门的使能端。这两个控制信号不能同时有效。

当接收控制信号有效时，左列的三态门打开，右列的三态门关闭，因而数据由右面传送到左面。

反之，当发送控制信号有效时，右列的三态门打开，左列的三态门关闭，因而数据由左面传送到右面。

数据通路

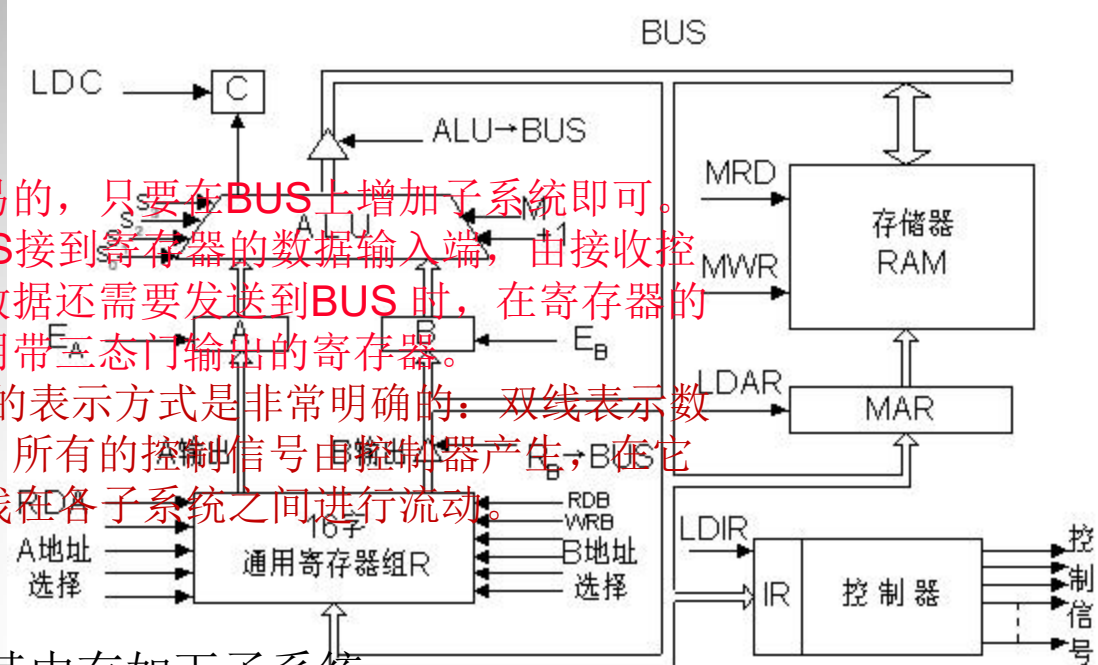
- 什么是数据通路

- 数字系统中,各个子系统通过数据总线连接形成的数据传送路径称为数据通路。
- 数据通路的设计直接影响到控制器的设计,同时也影响到数字系统的速度指标和成本。
- 一般来说,处理速度快的数字系统,它的独立传送信息的通路较多。但是独立数据传送通路一旦增加,控制器的设计也就复杂了。
- 因此,在满足速度指标的前提下,为使数字系统结构尽量简单,一般小型系统中多采用单一总线结构。在较大系统中可采用双总线或三总线结构。

数据通路结构

对单总线的系统来说，扩充是非常容易的，只要在**BUS**上增加子系统即可。例如增加一个寄存器时，可将总线**BUS**接到寄存器的数据输入端，由接收控制信号将数据打入。如果该寄存器的数据还需要发送到**BUS**时，在寄存器的输出端加上三态门即可，或者干脆使用带三态门输出的寄存器。

图中所示的数据通路中，两类信息的表示方式是非常明确的：**双线表示数据信息**，带箭头的单线表示控制信号。所有的控制信号由控制器产生，在它们的协调配合下，数据流通过**BUS**总线在各子系统之间进行流动。



数据通路结构

— 单总线结构的数据通路例子，其中有如下子系统：

- » **通用寄存器组R**：容量16个字，双端口输出。
- » **暂存器A和B**：保存通用寄存器组读出的数据或**BUS**上来的数据。
- » **算术逻辑单元ALU**：有S3、S2、S1、S0、M五个控制端，以选择运算类型。
- » **寄存器C**：保存ALU运算产生的进位信号。
- » **RAM随机读写存储器**：读/写操作受MRD/MWR控制信号控制。
- » **MAR**：RAM的专用地址寄存器，寄存器的长度决定RAM的容量。
- » **IR**：专用寄存器，可存放由RAM读出的一个特殊数据。
- » **控制器**：用来产生数据通路中的所有控制信号，它们与各个子系统上的使能控制信号一一对应。
- » **BUS**：单一数据总线，通过三态门与有关子系统连接。

微程序控制器的基本原理

- 微命令
 - 控制部件通过控制线向执行部件发出各种控制命令
- 微操作
 - 执行部件接受微命令所执行的操作
- 反馈信息
 - 执行部件通过反馈线向控制部件反映当前操作的结果
 - 控制部件根据执行部件的“状态”标志下达新的微命令
- 基本周期/机器周期
 - 几个时钟周期
- 微指令组成
 - 一组实现一定操作功能的微命令的组合
 - 读出微码指令与执行这条微码指令的时间总和
 - 一条微指令中包含若干个微命令
 - 分头并行地控制执行部件进行相应的微操作
 - 测试判别信息，实现控制算法流程图的条件分支
 - 下一条微指令的地址

微指令的典型结构

- 长条框内的符号×表示一个二进制位(bit)

- 操作控制

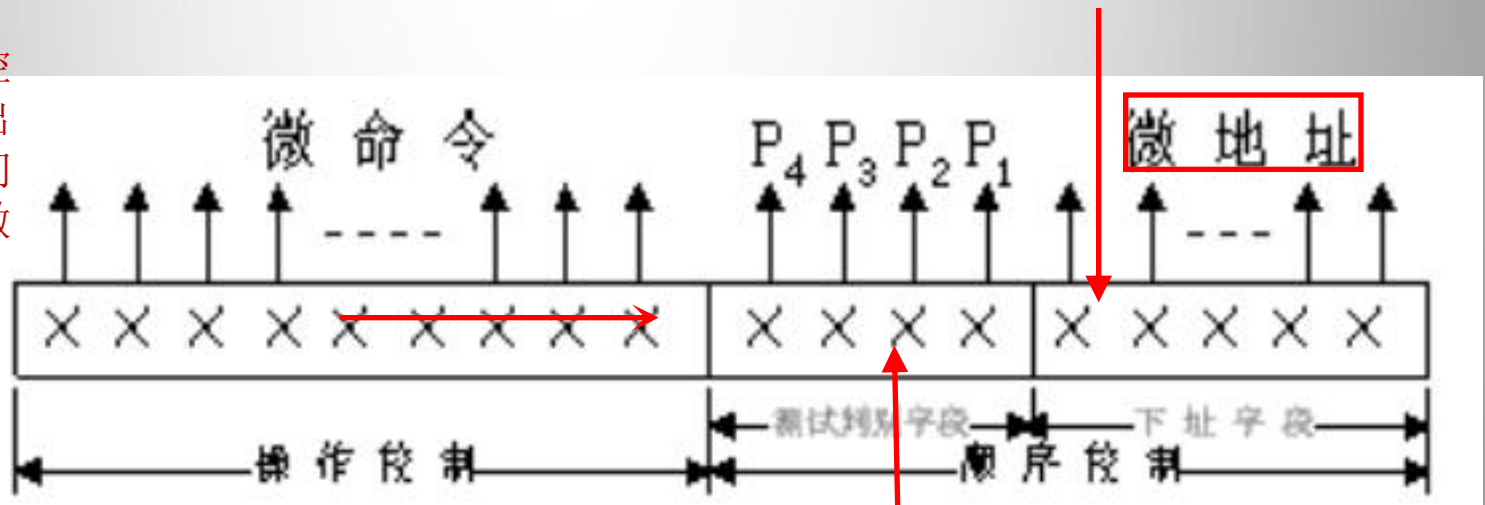
- 若干微命令

在系统的一个基本状态周期（又称机器周期）中，一组实现一定操作功能的微命令的组合，构成一条微指令。

- 测试判别信息

- 判别字段无效时，**下址字段信息**→下条微指令的地址
 - 判别字段有效时，根据反馈信息对下址字段信息进行修改，**修改后的地址**→下条微指令的地址

控制器通过一条条控制线向执行部件发出各种控制命令，我们把这些控制命令叫做微命令。



而执行部件接受微命令所执行的操作叫做微操作。

微指令的结构说明

- 强调两点

- 第一，一条微指令的有效持续时间是系统的一个基本周期，它表示从ROM中读出微指令与执行这条微指令的时间总和。当从ROM中读出下一条微指令后，当前的这条微指令即失效。
- 第二，一条微指令中包含若干个微命令，它们分头并行地控制执行部件进行相应的微操作。

- 测试判别信息

- 微指令除给出微命令信息外，还应给出测试判别信息。
- 一旦出现此信息，执行这条微指令时要对系统的有关标志进行测试，从而实现控制算法流程图中出现的条件分支。
- 微指令中还包含下一地址字段，该字段将指明ROM中下一条微指令的地址。

- 微指令的典型结构

- 其中微命令字段给出执行部件的控制信号：
 - » ×编码为**1**，表示有微命令，×编码为**0**表示无微命令。
- 测试判别字段和下一地址字段一起实现顺序控制：
 - » 当测试判别字段无效时（×编码为**0**），下址字段信息即是下条微指令的地址；
 - » 当判别测试字段有效时（其中一个×编码为**1**），根据执行部件反馈线上的标志信息对下址字段信息进行修改，修改好的地址即为下条微指令的地址。

微码程序

- 微程序

- 微程序是由若干条微指令组成的序列。
- 微程序相当于前述的流程图，

- 一条机器指令的功能

- 由若干条微指令组成的序列来解释和执行

- 机器执行一条指令的过程

- 执行一个相应的微程序的过程

- 微程序实质上是将控制器的控制算法变成了微程序流程图

- 一般数字系统而言，微程序实质上就是将控制算法流程图ASM用EPROM等来实现。

- 微码程序技术的革命

- 使大型复杂数字系统控制器的设计发生了革命性的变化。
- 微程序技术可代替硬件布线的控制技术

- » 存有控制代码的EPROM存储器，取代门电路和触发器等组成的硬件数字电路网络

- 通用性

微程序控制器的一般结构

- 微程序控制器的结构

- 微程序控制器的结构取决于微码指令的格式。
- 它由控制存储器、微地址寄存器、微命令寄存器和地址转移逻辑几部分组成。

- 控制存储器

- ROM中存放微码程序，也就是全部的微指令。
- ROM的容量取决于微指令的总数。

假如控制器需要128条微指令，则微地址寄存器长度为7位。ROM的字长取决于微指令长度。如果微指令为32位，则ROM的字长就是32位。实际应用中ROM可采用EPROM或E2PROM、EAROM，用户写入和修改微程序比较方便。

- 微指令寄存器

- 微地址寄存器和微命令寄存器两者的总长度即为一条微指令的长度
- 二者合在一起称为微指令寄存器。
- 微命令寄存器

- » 暂存由控制存储器中读出的当前微码指令的控制信息

- » 包括微命令控制字段和测试字段，可由8D寄存器组成。

- 微地址寄存器

- » 存放下一条微指令的（默认临时）地址

微地址寄存器暂存由控制存储器读出的当前微指令的下址字段信息。它可由带RD、SD强置端的D触发器组成。其中时钟端和D端配合用做ROM的读出打入，用SD进行下址修改。

- 地址转移逻辑

- 组合逻辑电路
- 条件转移：其输入是当前微指令的判别测试字段Pi、执行部件反馈的“状态条件”及时间因素T4。

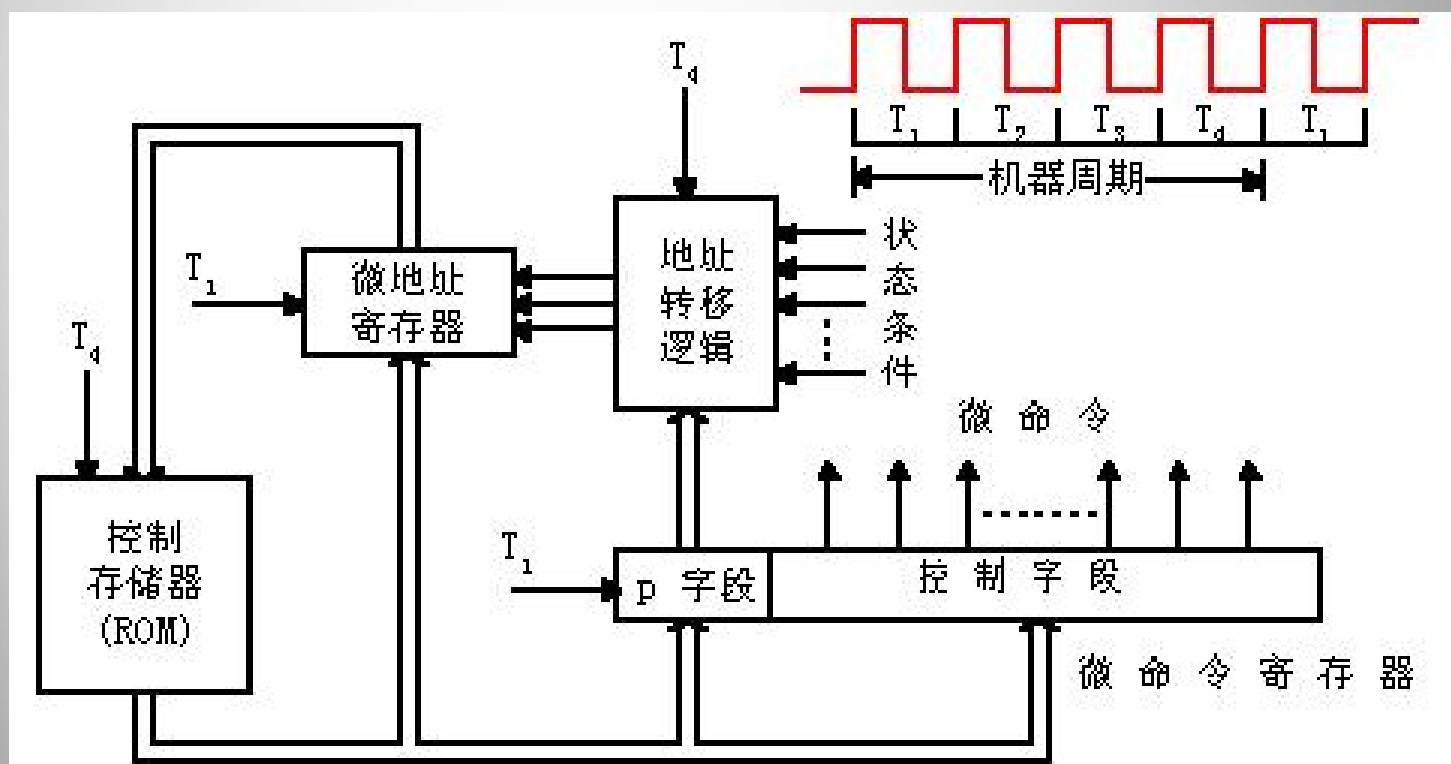
- 当微程序出现分支时修改微地址寄存器的内容，并按修改好的微地址读下一条

控制时序信号

一个基本机器周期中的**控制时序信号**

- 1) 用上一周期的**T₄**时间按微地址寄存器内容从ROM中读一条微指令
- 2) 经过一段时间后被读出，用当前周期的**T₁**时间存入微指令寄存器
- 3) **T₂, T₃**时间用来控制处理、控制执行部件进行操作，
- 4) **T₄**时间修改微地址寄存器中的内容，再读下一条微指令。

一个基本机器周期中的控制时序信号。



微程序控制器的设计

- 硬件设计
 - 选取存储器和寄存器，设计地址转移逻辑
- 微程序编制
 - 需要确定微程序流程图，即控制算法流程图。
- 状态化简和状态分配
 - 已无关紧要
- 状态最简
 - 勿需过多考虑
- 以微指令结构为基础
 - 可以编制任何所要求的微程序
 - 可长可短，灵活自如，且能实现多路并行分支
- 微程序流程图确定
 - 使用微指令格式将微程序转化为二进制代码

微程序控制器的设计步骤

- (1) 设计微程序

- 就是确定微程序流程图，也就是控制算法流程图。微程序流程图的一条微指令，相当于ASM流程图中的一个状态。
- 控制存储器中一个地址转移到下一个地址是随意的，灵活的。因此增加微指令或减少微指令不必改动硬件。能实现多路并行转移，而不受“状态分配”的限制。

- (2) 确定微指令格式

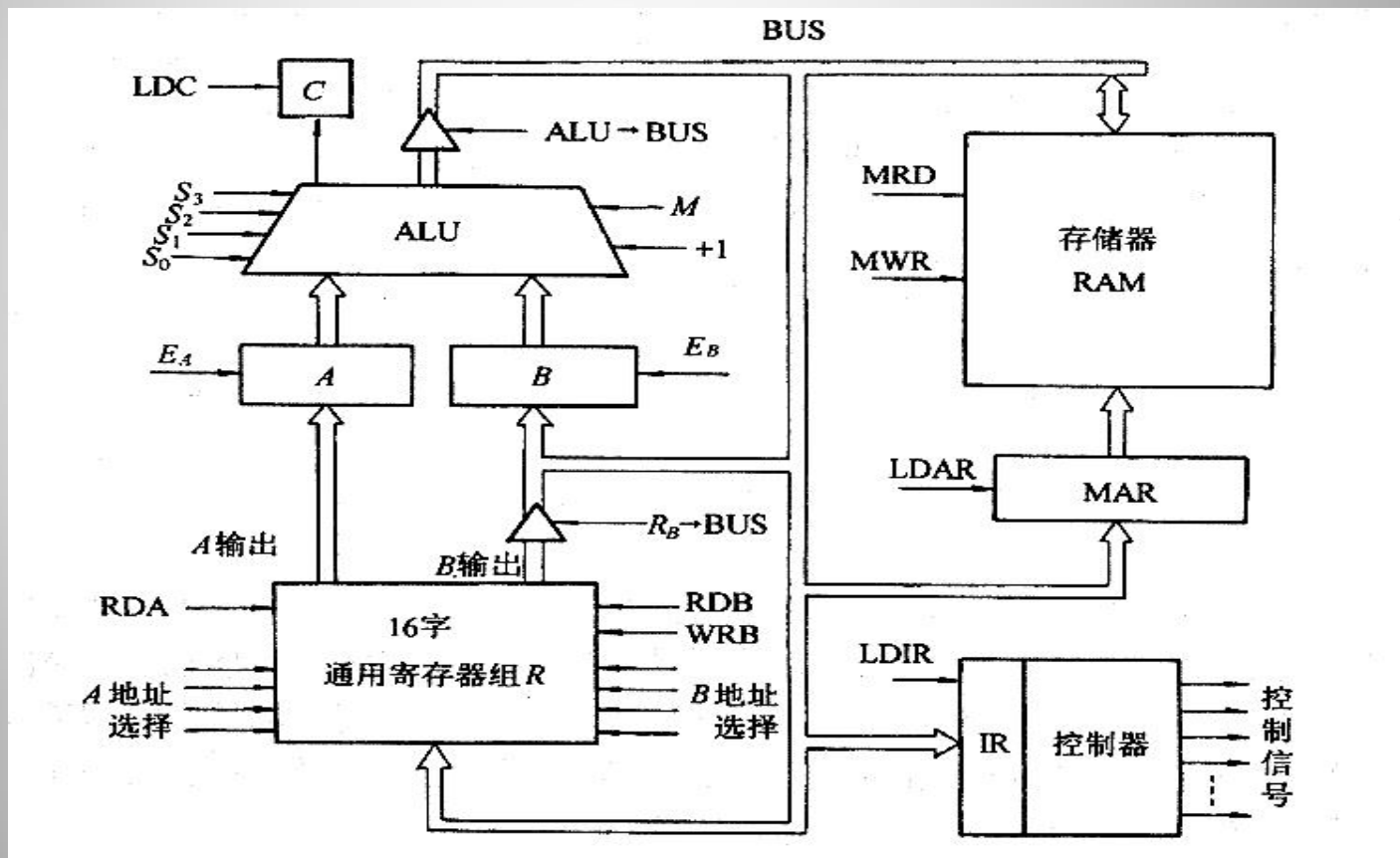
- 微指令格式中的操作控制字段取决于执行部件的子系统需要多少微指令。假定采用直接控制方式，执行部件需要10个微命令，则操作控制字段需要10位。
- 测试判别字段取决于微程序流程图中有多少处分支转移。假定有3处分支，则测试判别字段需要3位。
- 下址字段取决于微程序流程图的规模。假定微程序共用50条微指令，则下址字段至少需要6位。这是因为ROM地址译码时， $2^6=64$ ，6位地址可容纳64条微指令。

微程序控制器的设计步骤（2）

- **（3）将微程序编译成二进制代码**
 - 根据确定的微指令格式，将微程序流程图中的每一条指令编译成二进制代码，这项工作可由人工完成。
- **（4）微程序写入控制存储器**
 - 将二进制代码的微程序写入E2PROM中。
- **（5）设计硬件电路**
 - 硬件电路包括微地址寄存器、微命令寄存器和地址转移逻辑三部分。
 - 前两部分可选用适当的标准寄存器芯片（如74LS36、74LS273等）。
 - 地址转移逻辑的输入是测试判断标志 P_i 、状态条件和时间因素 T_j （读ROM时间），先写出逻辑表达式，然后用门电路芯片实现。

【例8.11】

- 按照图8.33数据通路，设计微指令格式，确定微程序控制器的基本方案。假定测试判别字段有2位，下址字段有4位。



数据通路和控制器（放大图）

RAM选用2114/2864型

三态缓冲器采用74LS244

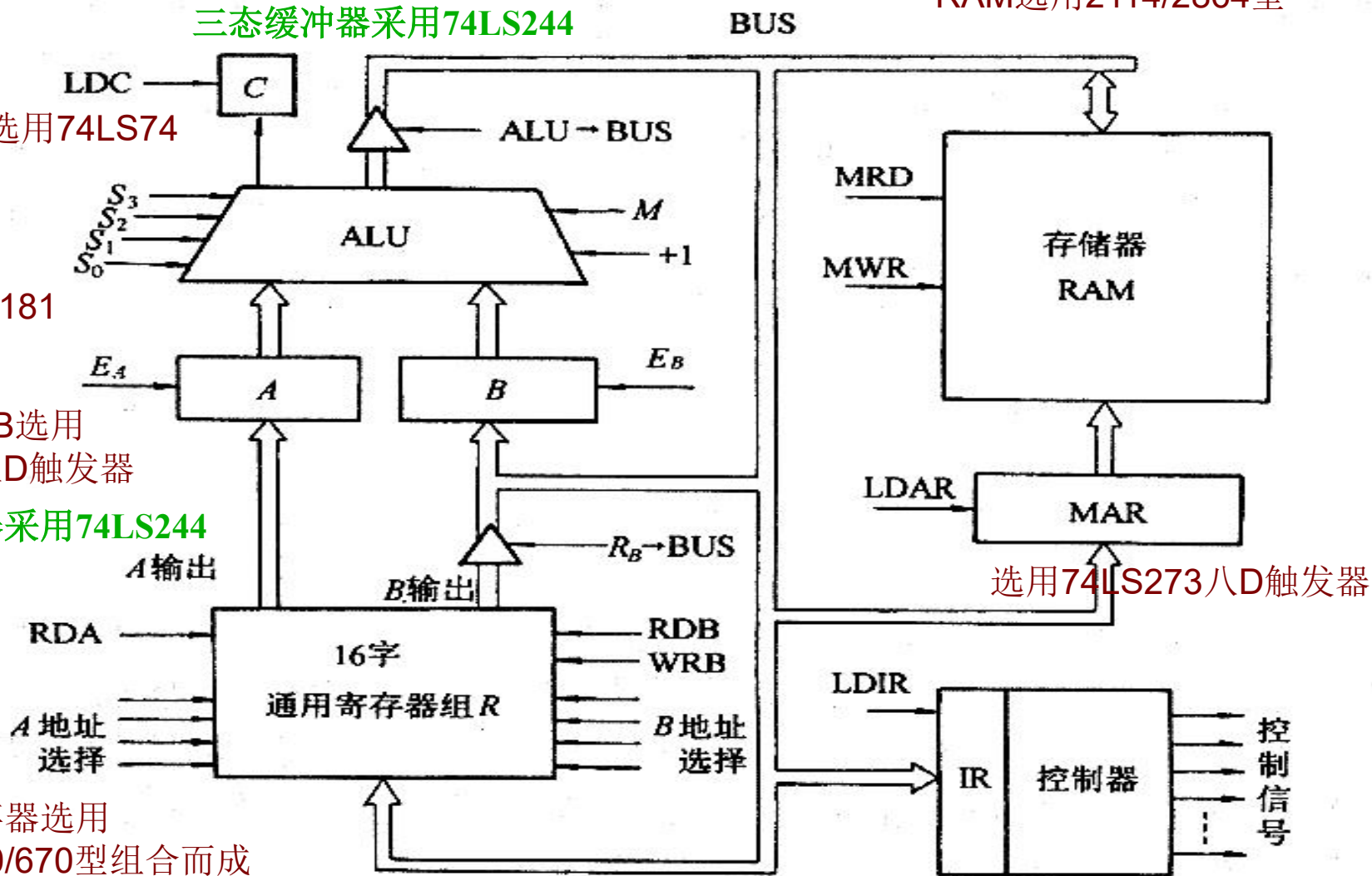
寄存器C选用74LS74

ALU=74LS181

寄存器A和B选用
74LS273八D触发器

三态缓冲器采用74LS244

通用寄存器选用
74LS170/670型组合而成



数据通路

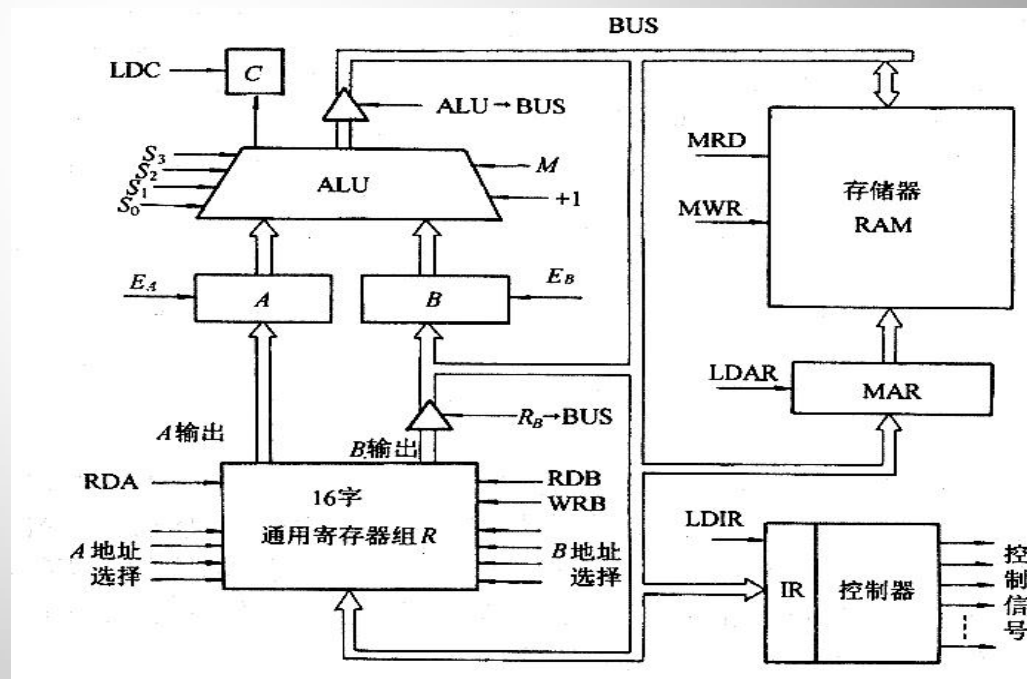
- 数据总线联结形成的数据传送路径

- 独立传送信息的通路数量→性能和复杂度

- 子系统

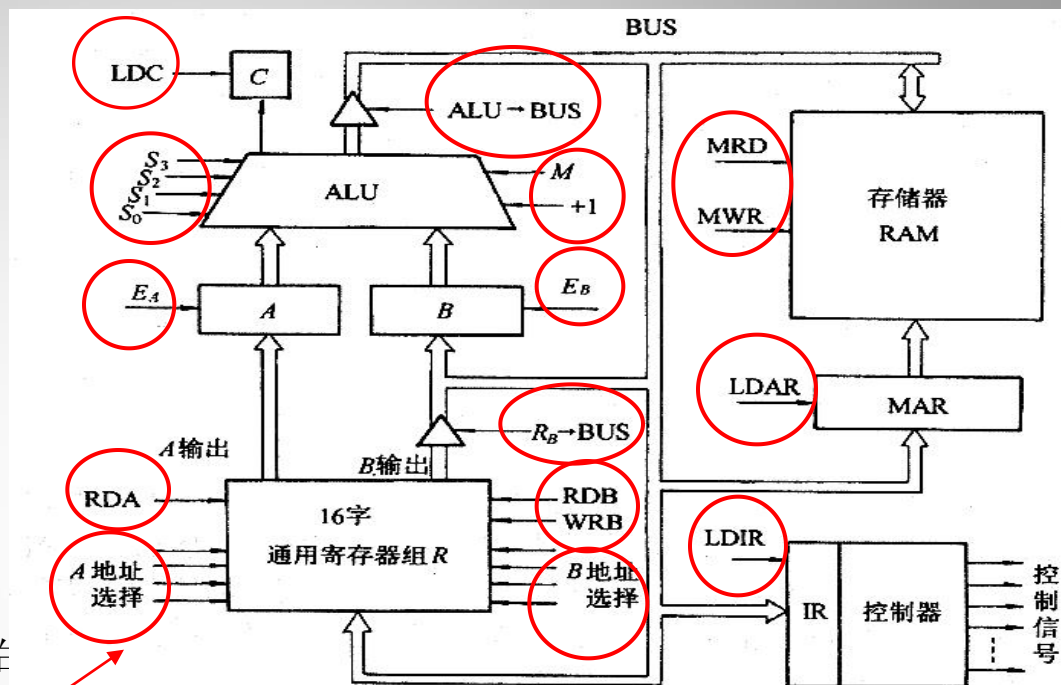
- 暂存器A,B
 - 16字的通用寄存器组
 - ALU
 - 存储器RAM
 - 地址寄存器MAR
 - 总线BUS和三态门

- 数据总线



数据通路分析

- 通用寄存器组
 - 双输出口
- 运算器
 - BUS
 - 寄存器组A输出总线
- BUS
 - 输出通过三态门连接
 - RAM本身带三态，双向
- 数据信息
- 控制信息



ALU算术运算单元

- 74LS181

- S0,S1,S2,S3功能选择端和M实现16种逻辑操作和算术运算
 - 加, 减, 与, 或, 异或, 取反等

- M工作方式选择

- 当工作方式控制端 (M) 为低电平时, 执行算术运算
 - 当工作方式控制端 (M) 为高电平时, 执行逻辑运算

- +1

- 进位输入

- C

- 进位输出

Function Table

Mode Select Inputs				Active LOW Operands & F _n Outputs		Active HIGH Operands & F _n Outputs	
				Logic (M = H)	Arithmetic (Note 2) (M = L) (C _n = L)	Logic (M = H)	Arithmetic (Note 2) (M = L) (C _n = H)
S3	S2	S1	S0				
L	L	L	L	$\bar{A}$	A minus 1	$\bar{A}$	A
L	L	L	H	$\overline{AB}$	AB minus 1	$\bar{A} + \bar{B}$	A + B
L	L	H	L	$\bar{A} + \bar{B}$	$\overline{AB}$ minus 1	$\bar{A} B$	A + $\bar{B}$
L	L	H	H	Logic 1	minus 1	Logic 0	minus 1
L	H	L	L	$\bar{A} + \bar{B}$	A plus (A + $\bar{B}$)	$\overline{AB}$	A plus $\overline{AB}$
L	H	L	H	$\bar{B}$	AB plus (A + $\bar{B}$)	$\bar{B}$	(A + B) plus $\overline{AB}$
L	H	H	L	$\bar{A} \oplus \bar{B}$	A minus B minus 1	A $\oplus$ B	A minus B minus 1
L	H	H	H	A + $\bar{B}$	A + $\bar{B}$	$\overline{AB}$	AB minus 1
H	L	L	L	$\bar{A} B$	A plus (A + B)	$\bar{A} + B$	A plus AB
H	L	L	H	A $\oplus$ B	A plus B	$\bar{A} \oplus \bar{B}$	A plus B
H	L	H	L	B	$\overline{AB}$ plus (A + B)	B	(A + $\bar{B}$) plus AB
H	L	H	H	A + B	A + B	AB	AB minus 1
H	H	L	L	Logic 0	A plus A (Note 1)	Logic 1	A plus A (Note 1)
H	H	L	H	$\overline{AB}$	AB plus A	A + $\bar{B}$	(A + B) plus A
H	H	H	L	AB	$\overline{AB}$ minus A	A + B	(A + $\bar{B}$) plus A
H	H	H	H	A	A	A	A minus 1

4类数据操作

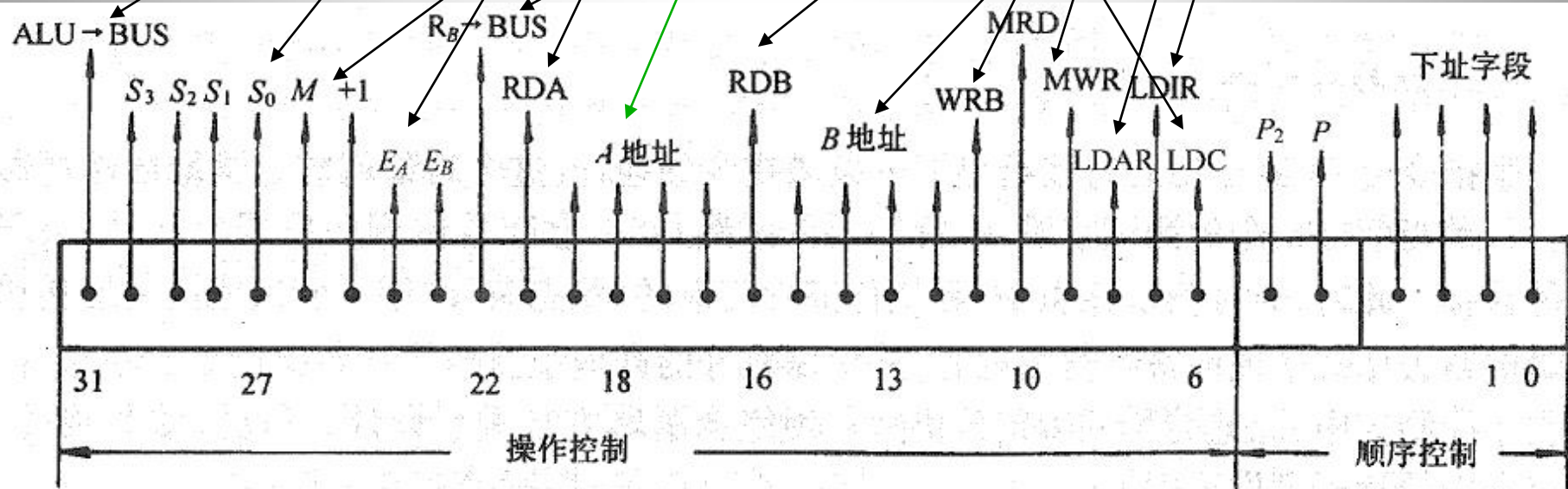
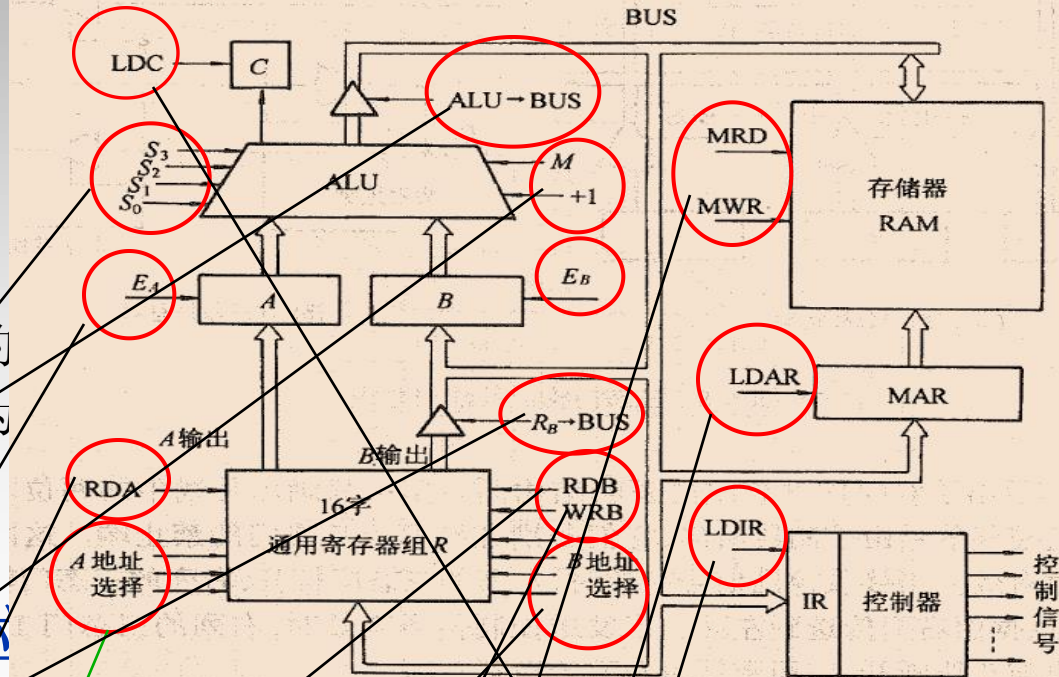
- **1) $R_i + R_j \rightarrow R_j$**
 - $RDA \rightarrow$ 以A地址读寄存器堆
 - $RDB \rightarrow$ 以B地址读寄存器堆
 - $WRB \rightarrow$ 以B地址写寄存器堆
- **2) $R_j \rightarrow RAM$**
 - $LDAR \rightarrow$ 暂存器B为RAM地址打入MAR
 - $RDB + MWR \rightarrow$ 写入RAM
- **3) $RAM \rightarrow R_j$**
 - $MRD \rightarrow$ 读出RAM + WRB 写入R
- **4) $R_i + RAM \rightarrow RAM$**
 - $RDA + MRD(\text{地址已有}) + MWR$

微指令格式

• 所有的控制信号

- 均标注在逻辑子系统的
- 因此微命令字段长度为
- 加上测试判别字段2位
- 微地址字段4位

• 微指令长度共计32位



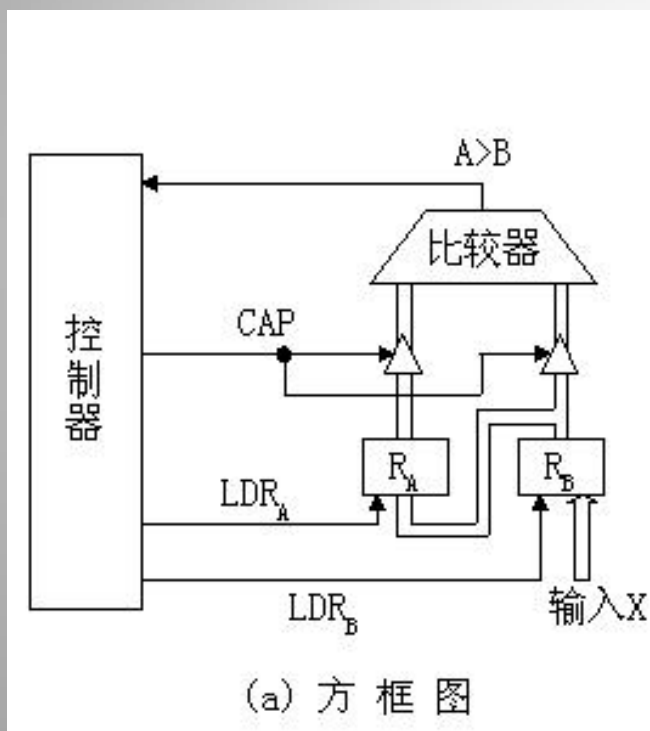
根据微指令格式，确定EPROM的容量为16个存储单元，字长32位。微地址寄存器4位，微命令寄存器28位。

【例8.12】

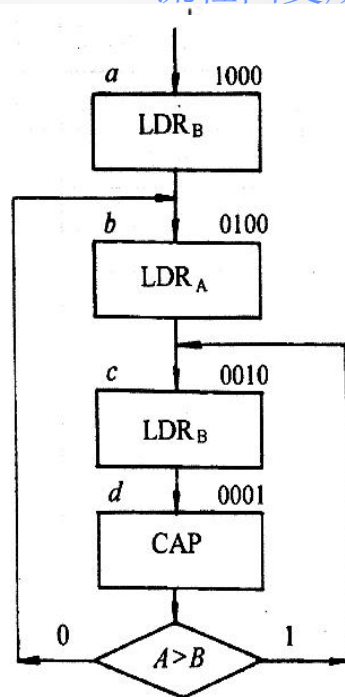
- 数字比较系统硬件框图及ASM流程图如图8.26所示，请设计其微程序控制器。

解

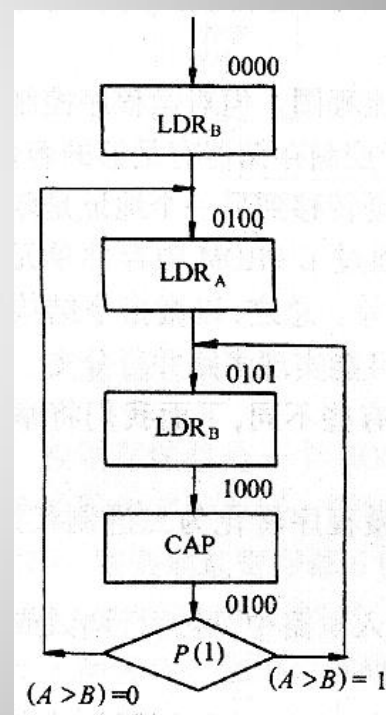
ASM流程图变成微程序流程图



硬件框图



ASM流程图



微程序流程图

解

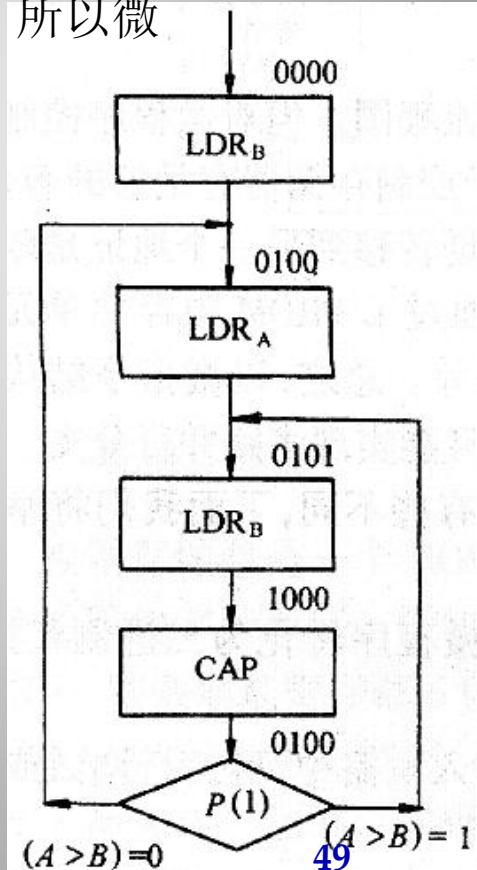
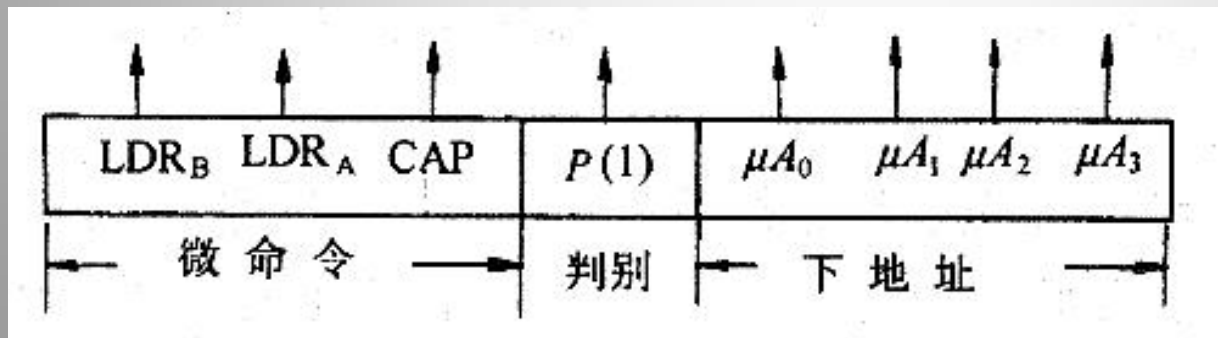
• (1) 确定地址

– 每一个状态框变成一条微指令

– 四条微指令，令其地址为0000，0100，0101，1000，它们是任意安排的，用4位二进制码表示的微地址。按4位地址推算EPROM的存储容量为 $2^4=16$ 个存储单元。

• (2) 确定微指令格式

- 本系统中的控制信号只有3个（LDR_A、LDR_B，CAP），所以微命令字段为3位；
- 加上判别测试字段1位；
- 下地址字段4位；
- 故微指令字长度为8位。



微指令执行

- (3) 确定地址转移逻辑表达式
 - 分支转移
 - » 第4条微指令执行后出现
 - » 判别测试框(第4条微指令)
 - 判别测试标志P(1)
 - P(1)=1, 根据转移条件(A>B)来决定下条微指令
 - » (A>B)=0, 执行第2条微指令(微地址0100)
 - » (A>B)=1, 执行第3条微指令(微地址0101)。
 - 这两条微指令地址的差别仅在于最低位上是1还是0
 - » 因此地址转移逻辑表达式为:

$$\mu A_3 = P(1) \cdot (A > B) \quad \mu A_3 = P1 \cdot (A > B) \cdot T3$$

- » 其中: A>B是比较器的输出信号, (A>B)=1表示A数大于B数, (A>B)=0表示A数小于等于B数。
- » 用机器周期的T3节拍修改微地址并读EPROM, 再用下一周期的T1节拍打入到微指令寄存器中。
- 当微程序出现分支时
 - 下个微地址的安排要考虑修改的方便性
 - 注, 微程序不出现分支时
 - » 下个微地址, 在E²PROM存储容量的范围内可任意设计

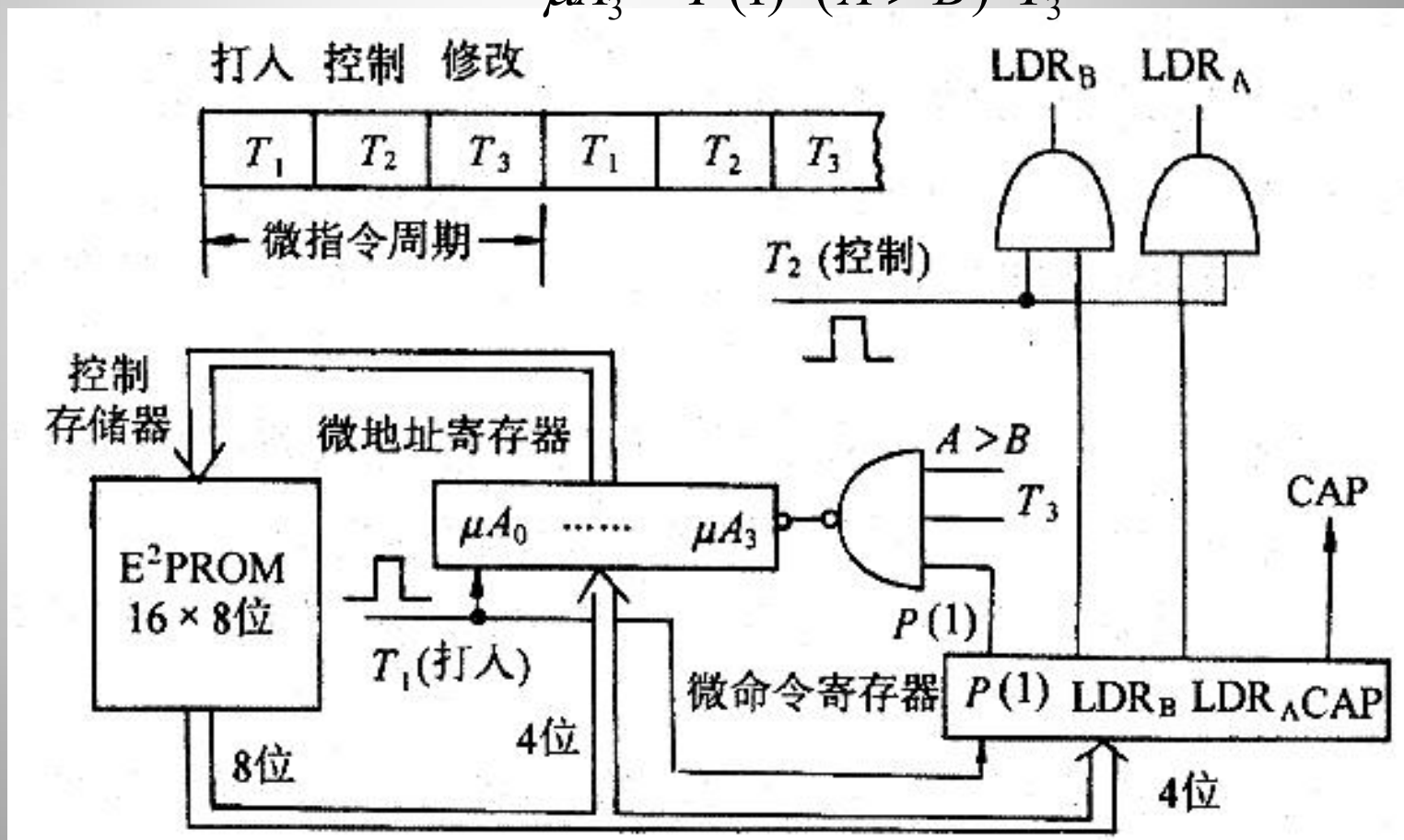
(4) 硬件电路

• 微地址寄存器采用带有强置端的双D触发器

- 通过 μA_3 触发器 S_D 端置“1”的方法可将其修改为1
- 其他3个触发器不做修改

其中LDRB、LDRA是脉冲控制信号（持续时间 T_2 ），CAP是电位控制信号（持续时间一个机器周期）。

$$\mu A_3 = P(1) \cdot (A > B) \cdot T_3$$



(5) 微程序

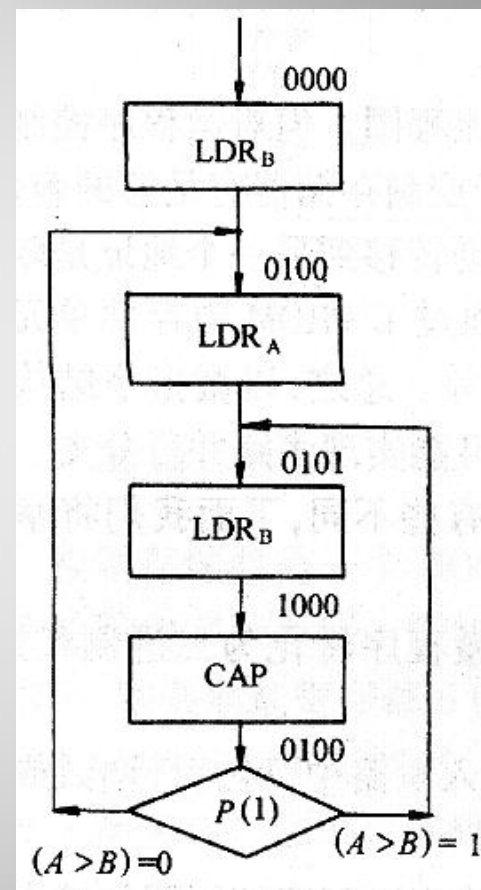
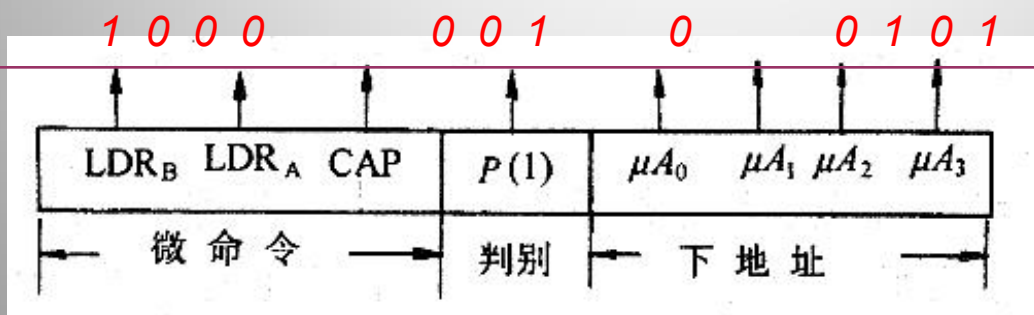
在微程序无分支的情况下，下地址可按EPROM容量随意填写，但必须限制在微地址寄存器指定的范围内，**且不允许两条微指令使用同一个微地址。**

控制存储器E²PROM中存放由4条微指令组成的微程序，它们是按微指令格式转换的二进制代码

表8.7 微程序代码

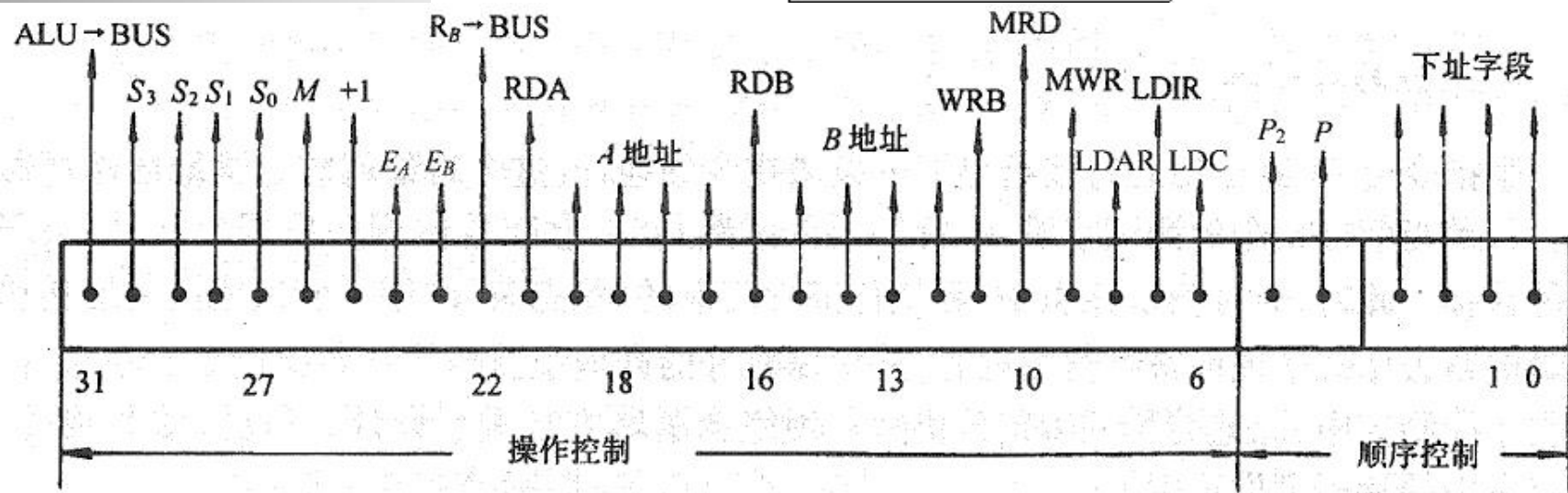
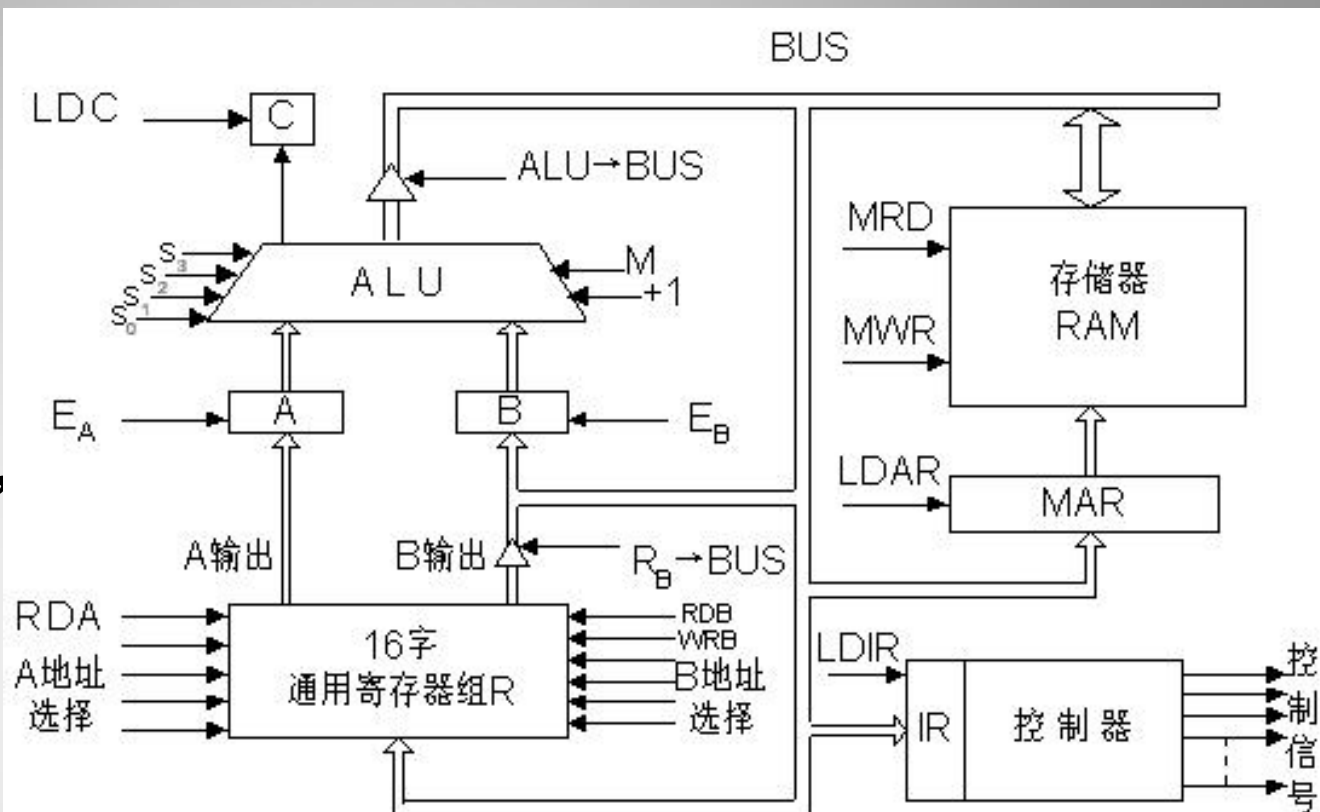
当前微地址 (存储器位置)	微指令二进制代码		
	微命令	判别	下地址
0 0 0 0	1 0 0	0	0 1 0 0
...	X X X	X	X X X X
0 1 0 0	0 1 0	0	0 1 0 1
0 1 0 1	1 0 0	0	1 0 0 0
...	X X X	X	X X X X
1 0 0 0	0 0 1	1	0 1 0 0

不能



【例8.13】

采用例8.11的数据通路和图8.32所示的微指令格式，编制如下四种基本操作的微程序流程图，并完成微地址转移逻辑的设计。



【例8.13】

- 采用例8.11的数据通路和图8.32所示的微指令格式，编制如下四种基本操作的微程序流程图，并完成微地址转移逻辑的设计。
- 00 $RAM \rightarrow Rj(2)$ (从RAM中取数至R2)
- 01 $Ri(3) \text{ 加 } Rj(4) \rightarrow Rj(4)$ (R3, R4中二数算术加)
- 10 $Ri(6) \rightarrow RAM$ (R6中的数存入RAM)
- 11 $Ri(7) \oplus Rj(8) \rightarrow Rj(8)$ (R7, R8中二数逻辑异)
- 假设条件：
 - (1)四种基本操作分别用4个状态的编码(00, 01, 10, 11)来表示，该编码值由IR0和IR1两个触发器组成的计数器提供。对应计数器的一种状态，则执行相应的一种基本操作；
 - (2)每当一种基本操作结束时，计数器进行加数而改变状态。但操作开始前计数器的初始状态是随机的，这意味着四种操作执行的顺序可认为是随意的，但时间上必须是连续的；
 - (3)通用寄存器的选择也是随意的；
 - (4)向RAM中取数、存数的地址均由暂存器B给出。

解

分析：（1）微程序设计

最上面的第一条入口地址为0000，这通过系统启动时将微地址寄存器清零来给出。

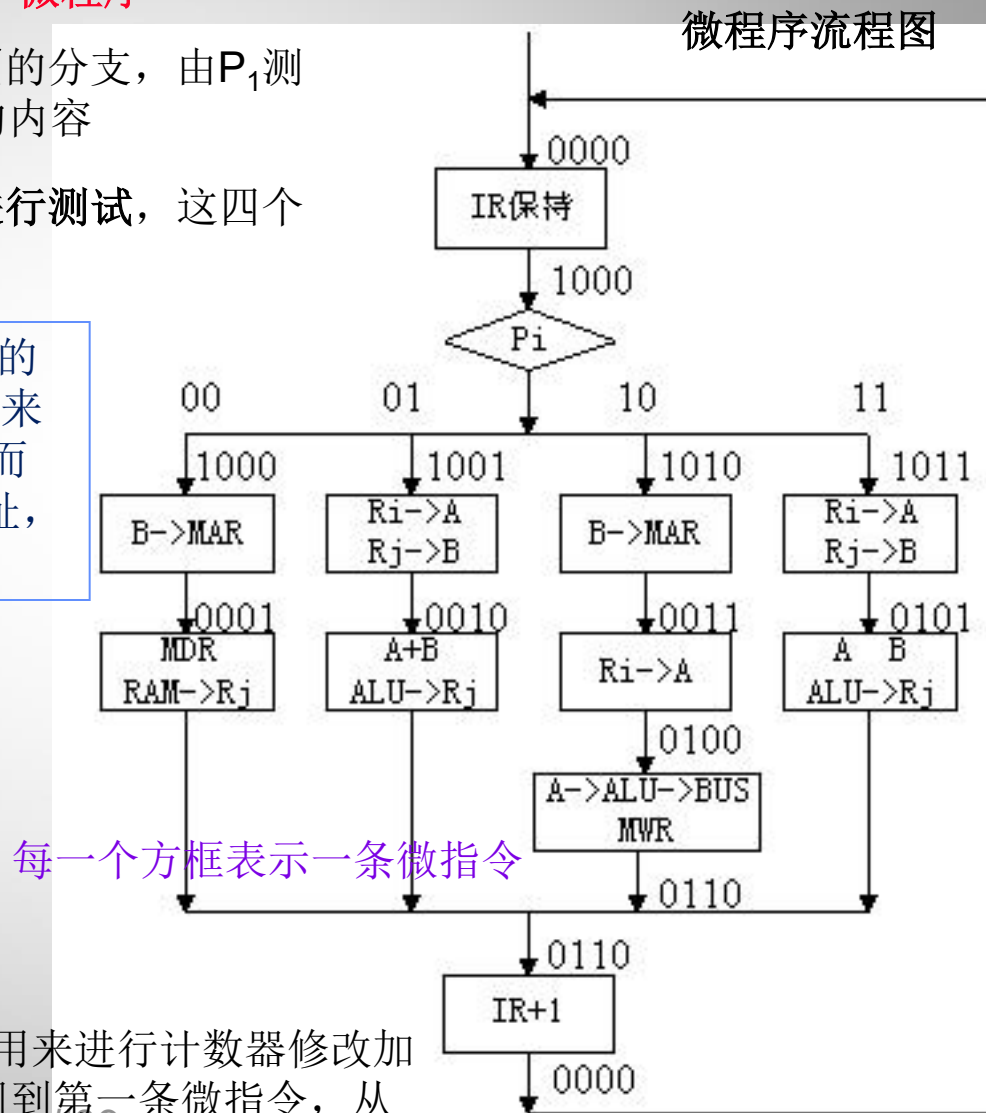
1) 对应四种基本操作，我们需要设计4个微程序

2) 具体执行某种操作前，微程序流程图的分支，由 P_1 测试来完成，测试的状态条件是 IR_0 ， IR_1 的内容

3) 四个基本操作执行过程中不需要再进行测试，这四个微程序不再出现分支情况。

第一条微指令的下地址1000是一个待修改的地址，在 P_1 测试时，根据 IR_0 、 IR_1 的状态来修改微地址寄存器的最后两位触发器，从而得出1000、1001、1010、1011四个微地址，实现了微程序的四路并行转移。

右上角的二进制码表示：当前微指令的地址；
右下角的数字表示：下条微指令的地址(在微程序无分支情况下) 或待修改的地址(在微程序有分支情况下)。



最下面一条微指令是一条公用微指令，用来进行计数器修改加1。由于它的下地址是0000，因而又返回到第一条微指令，从而实现了微程序的连续运行。

分析：（2）微地址转移逻辑设计

- 每条微指令的框内表示数据通路的选择操作
 - 是用微指令的操作控制字段发出的微命令来实现的。
- 从时间上讲，每条微指令的持续时间均为一个机器周期
 - (本例中由T1、T2、T3、T4四个节拍时间组成，4个时钟周期组成一个机器周期)。
 - 每一条微指令只完成一个机器周期时间所允许的微操作。
 - 而几条微指令组合在一起，可完成更多的操作。可完成课题任务所要求的基本操作。
- 微地址转移逻辑用两个三输入与非门实现

$$\mu A_3 = \overline{P_1 \cdot IR_0 \cdot T_4}$$

$$\mu A_4 = \overline{P_1 \cdot IR_1 \cdot T_4}$$

其中

P1为微指令的测试判别标志，
IR0，IR1为计数器二触发器的状态值，
T4表示时钟周期时间。

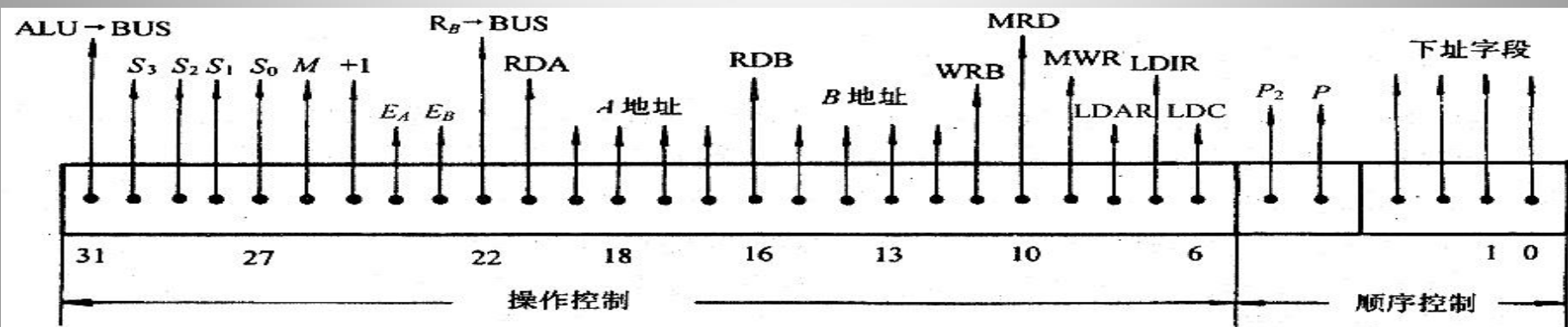
表达式说明，当P₁测试时，在T₄时刻按计数器IR₀、IR₁的内容修改 μAR₃和 μAR₄，并按修改了的微地址寄存器内容读出下条微指令。

(3) 微程序编译成二进制代码

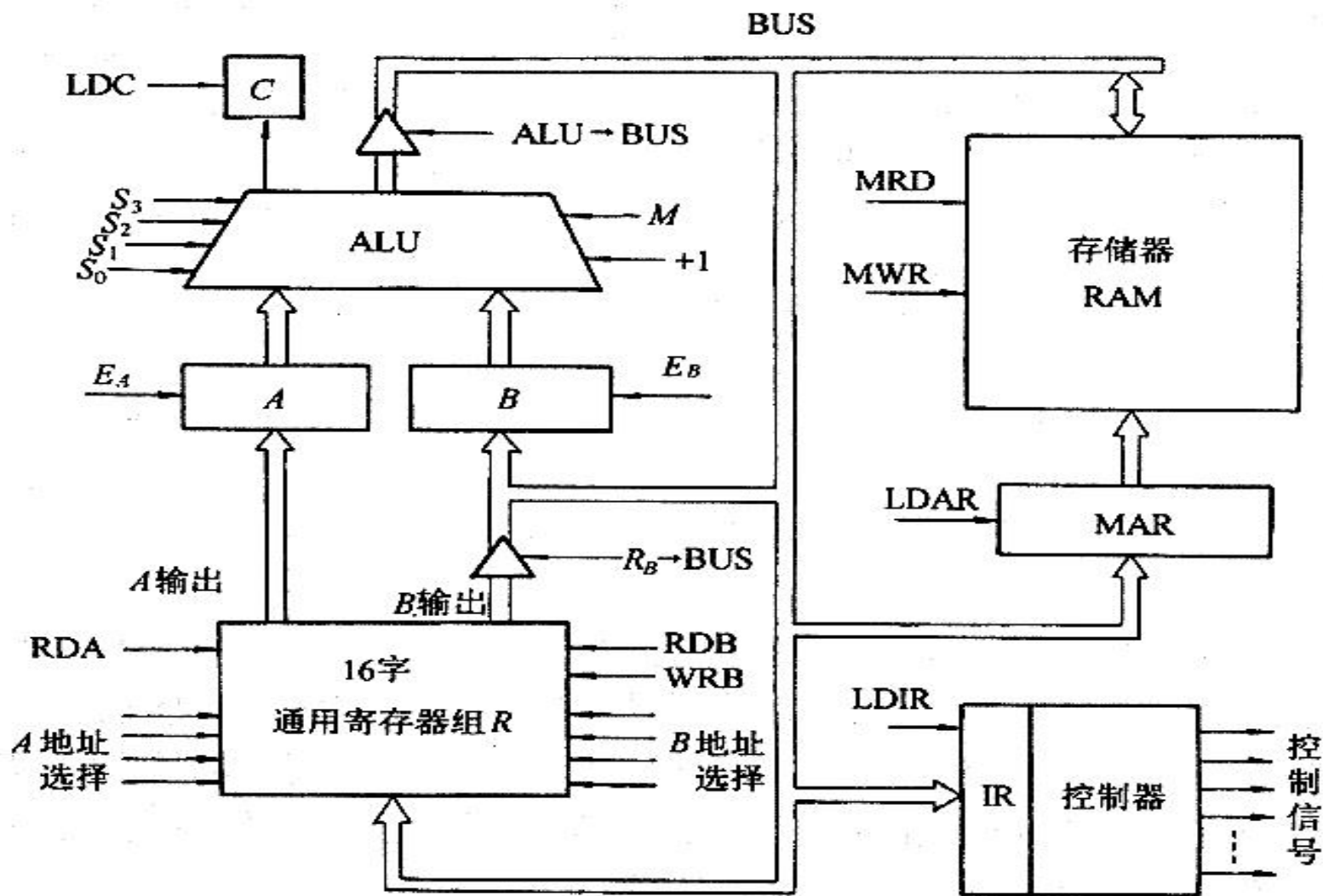
当前微地址	八进制表示的微命令	P_2P_1	下一微地址
0 0 0 0	0 0 0 0 0 0 0 0 0	0 1	1 0 0* 0*
0 0 0 1	0 0 0 0 0 0 2 6 0	0 0	0 1 1 0
0 0 1 0	3 1 0 0 0 0 4 4 0	0 0	0 1 1 0
0 0 1 1	0 0 1 1 3 0 0 0 0	0 0	0 1 0 0
0 1 0 0	3 7 4 0 0 0 0 1 0	0 0	0 1 1 0
0 1 0 1	2 6 0 0 0 1 0 4 0	0 0	0 1 1 0
0 1 1 0	0 0 0 0 0 0 0 0 2	0 0	0 0 0 0
...	...	...	...
1 0 0 0	3 2 4 0 0 0 0 0 4	0 0	0 0 0 1
1 0 0 1	0 0 1 7 1 6 4 0 0	0 0	0 0 1 0
1 0 1 0	3 2 4 0 0 0 0 0 4	0 0	0 0 1 1
1 0 1 1	0 0 1 7 3 7 0 0 0	0 0	0 1 0 1

不允许两条微指令使用同一个微地址。因为它是物理空间，住不下2个指令。

指令格式:



数据通路和控制器

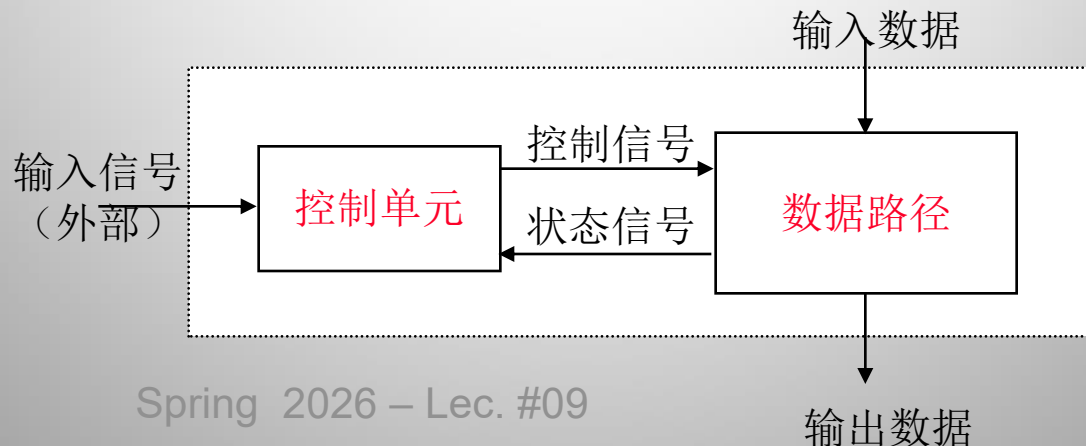


小结

- 微程序控制的主要优点
 - 是规范和灵活
 - 易于实现多路并行分支
 - 易于系统扩展
- 系统越复杂，越宜适用微程序控制方法进行设计
 - 只不过是微指令的重复应用
 - EPROM只不过是增加了一些存储单元
 - 而微指令寄存器并不改变
- **缺点:**
- 对一些非常简单的系统来说
 - 采用微程序控制方法从成本上来讲不一定合算。
- 从速度上讲
 - 微程序控制器的工作速度不如硬布线逻辑控制器快。

课堂测验（课后做）

- 如何计量寄存器中1的个数，采用数据选择器法实现控制器。画出数字系统电路框图和ASM图。
 - 设计的数字系统包括两个寄存器R1和R2，以及一个触发器E。系统对1的个数进行计量，将计数值预置到寄存器R1，设置寄存器R2等于这个数值。
 - 控制器的外部输入信号Start启动状态机的操作，Ready为状态机的状态指示。控制器采用异步复位，时钟正边沿触发。
 - 控制器还接收两个从数据路径来的状态输入E和Zero。这些信号表明最高有效位的寄存器数据是0。E是触发器输出，Zero是检测寄存器R1是否全0的结果输出。当R1等于0（即R1中不包含1）时，输出Zero = 1。



Microprogramming (课后参考)

How to organize the control signals

Implement control signals by storing 1's and 0's in a ROM

Horizontal vs. vertical microprogramming

Horizontal: 1 ROM output for each control signal

Vertical: encoded control signals in ROM, decoded externally
some mutually exclusive signals can be combined
helps reduce ROM length

Microprogramming

Register Transfer/Microoperations

14 Register Transfer operations become 22 Microoperations:

PC → ABUS

IR → ABUS

MBR → ABUS

RBUS → AC

AC → ALU A

MBUS → ALU B

ALU ADD

ALU PASS B

MAR → Address Bus

MBR → Data Bus

ABUS → IR

ABUS → MAR

Data Bus → MBR

RBUS → MBR

MBR → MBUS

0 → PC

PC + 1 → PC

ABUS → PC

Read/Write

Request

AC → RBUS

ALU Result → RBUS

Horizontal Microprogramming (参考)

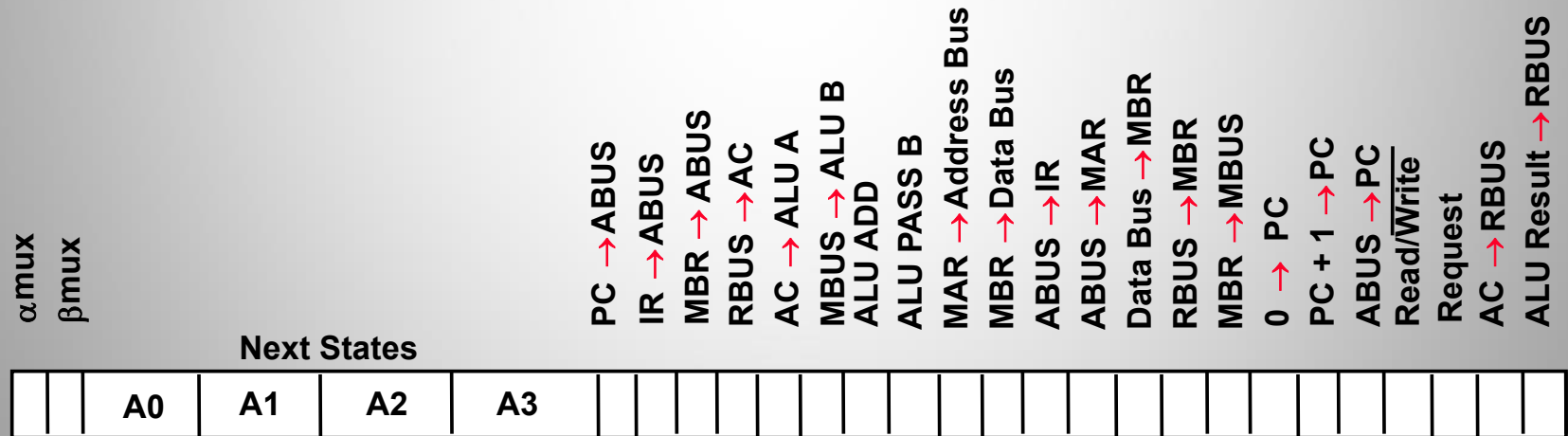
Horizontal Branch Sequencer

α, β Mux bits

4 x 4 Next State bits

22 Control operation bits

40 bits total



Horizontal Microprogramming

Moore Processor ROM

Moore Processor ROM

Current State (Address)	α_{mux} β_{mux}	Next States																									
		A0	A1	A2	A3	PC $\rightarrow$ ABUS	IR $\rightarrow$ ABUS	MBR $\rightarrow$ ABUS	RBUS $\rightarrow$ AC	AC $\rightarrow$ ALU A	MBUS $\rightarrow$ ALU B	ALU ADD	ALU PASS B	MAR $\rightarrow$ Address Bus	MBR $\rightarrow$ Data Bus	ABUS $\rightarrow$ IR	ABUS $\rightarrow$ MAR	Data Bus $\rightarrow$ MBR	RBUS $\rightarrow$ MBR	MBR $\rightarrow$ MBUS	0 $\rightarrow$ PC	PC + 1 $\rightarrow$ PC	ABUS $\rightarrow$ PC	Read/Write Request	AC $\rightarrow$ RBUS	ALU Result $\rightarrow$ RBUS	
RES (0000)	0 0	0001	0001	0001	0001	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0
IF0 (0001)	0 0	0010	0010	0010	0010	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0
IF1 (0010)	0 0	0010	0010	0011	0011	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
IF2 (0011)	0 0	0100	0100	0011	0011	0	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	1	1	0
IF3 (0100)	0 0	0100	0100	0101	0101	0	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0
OD (0101)	1 1	0110	1001	1011	1110	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
LD0 (0110)	0 0	0111	0111	0111	0111	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
LD1 (0111)	0 0	1000	1000	0111	0111	0	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	1	1	0
LD2 (1000)	0 0	0001	0001	0001	0001	0	0	0	1	0	1	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	1
ST0 (1001)	0 0	1010	1010	1010	1010	0	1	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	1	0
ST1 (1010)	0 0	0001	0001	1010	1010	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	1	0
AD0 (1011)	0 0	1100	1100	1100	1100	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0
AD1 (1100)	0 0	1101	1101	1100	1100	0	0	0	0	0	0	0	0	1	0	0	0	0	1	0	0	0	0	0	1	1	0
AD2 (1101)	0 0	0001	0001	0001	0001	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1
BR0 (1110)	0 1	0001	1111	0001	1111	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
BR1 (1111)	0 0	0001	0001	0001	0001	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0

Alpha inputs: 0 = Wait, 1 = IR<15>

Beta inputs: 0 = AC<15>, 1 = IR<14>

Horizontal Microprogramming

Advantages:

most flexibility -- complete parallel access to datapath control points

Disadvantages:

very long control words -- 100+ bits for real processors

NOTE: Not all microoperation combinations make sense!

Output Encodings:

Group mutually exclusive signals
Use external logic to decode

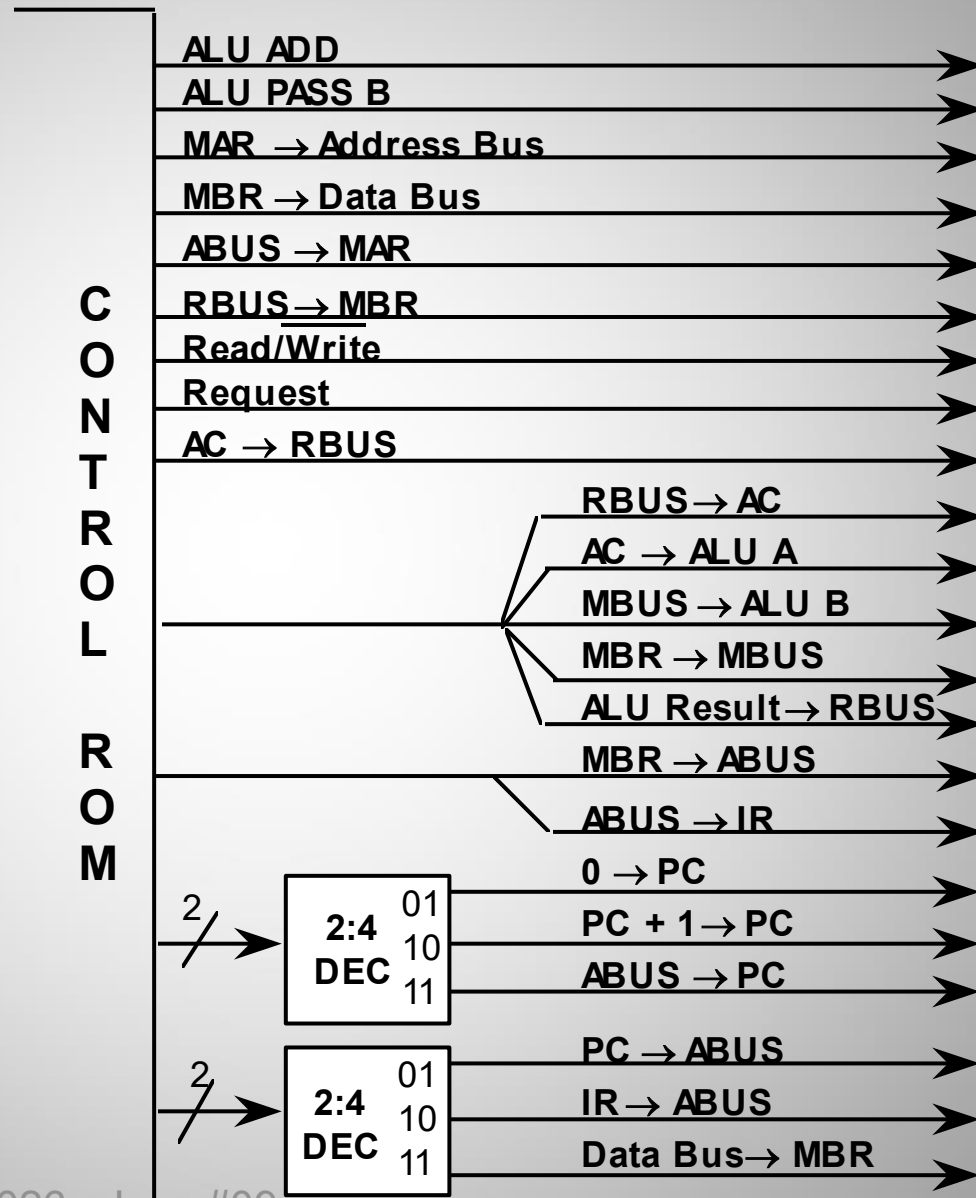
Example:

0 PC, PC + 1 PC, ABUS PC mutually exclusive

Save ROM bit with external 2:4 Decoder

Horizontal Microprogramming

Partially Encoded
Control Outputs



Vertical Microprogramming (参考)

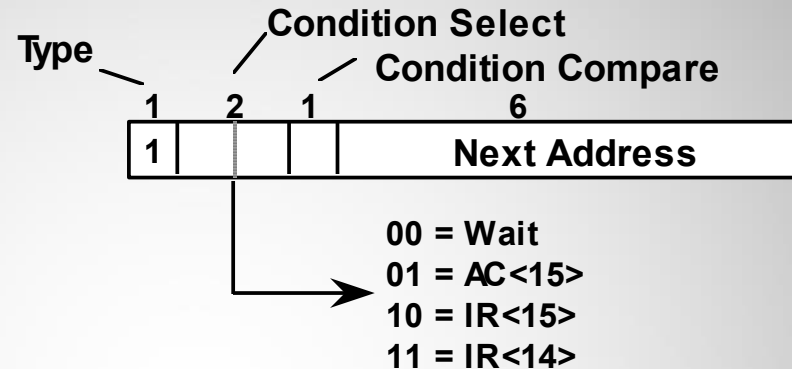
More extensive encoding to reduce ROM word length

- Typically use multiple microword formats:
 - Horizontal microcode -- next state + control bits in same word
 - Separate formats for control outputs and "branch jumps"
 - may require several microwords in a sequence to implement same function as single horizontal word
- **In the extreme, very much like assembly language programming**

Vertical Microprogramming

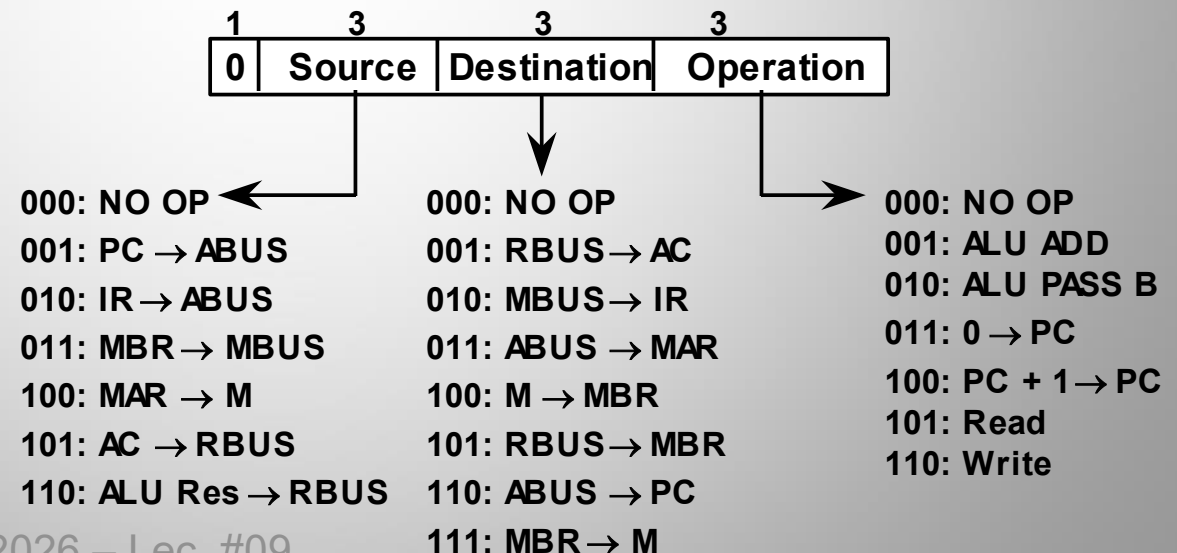
Branch Jump
Compare indicated
signal to 0 or 1

Branch Jump Format



Register Transfer
Source,
Destination,
Operation

Register Transfer Format



10 ROM Bits

Vertical Microprogramming

ROM ADDRESS	SYMBOLIC CONTENTS			BINARY CONTENTS			
000000	RES	RT	PC → MAR, PC +1 → PC	0	001	011	100
000001	IF0	RT	MAR → M, Read	0	100	000	101
000010		BJ	Wait=0, IF0	1	000	000	001
000011	IF1	RT	MAR → M, M → MBR, Read	0	100	100	101
000100		BJ	Wait=1, IF1	1	001	000	011
000101	IF2	RT	MBR → IR	0	011	010	000
000110		BJ	Wait=0, IF2	1	000	000	101
000111		RT	IR → MAR	0	010	011	000
001000	OD	BJ	IR<15>=1, OD1	1	101	010	101
001001		BJ	IR<14>=1, ST0	1	111	010	000
001010	LD0	RT	MAR → M, Read	0	100	000	101
001011	LD1	RT	MAR → M, M → MBR, Read	0	100	100	101
001100		BJ	Wait=1, LD1	1	001	001	011
001101	LD2	RT	MBR → AC	0	110	001	010
001110		BJ	Wait=0, RES	1	000	000	000
001111		BJ	Wait=1, RES	1	001	000	000

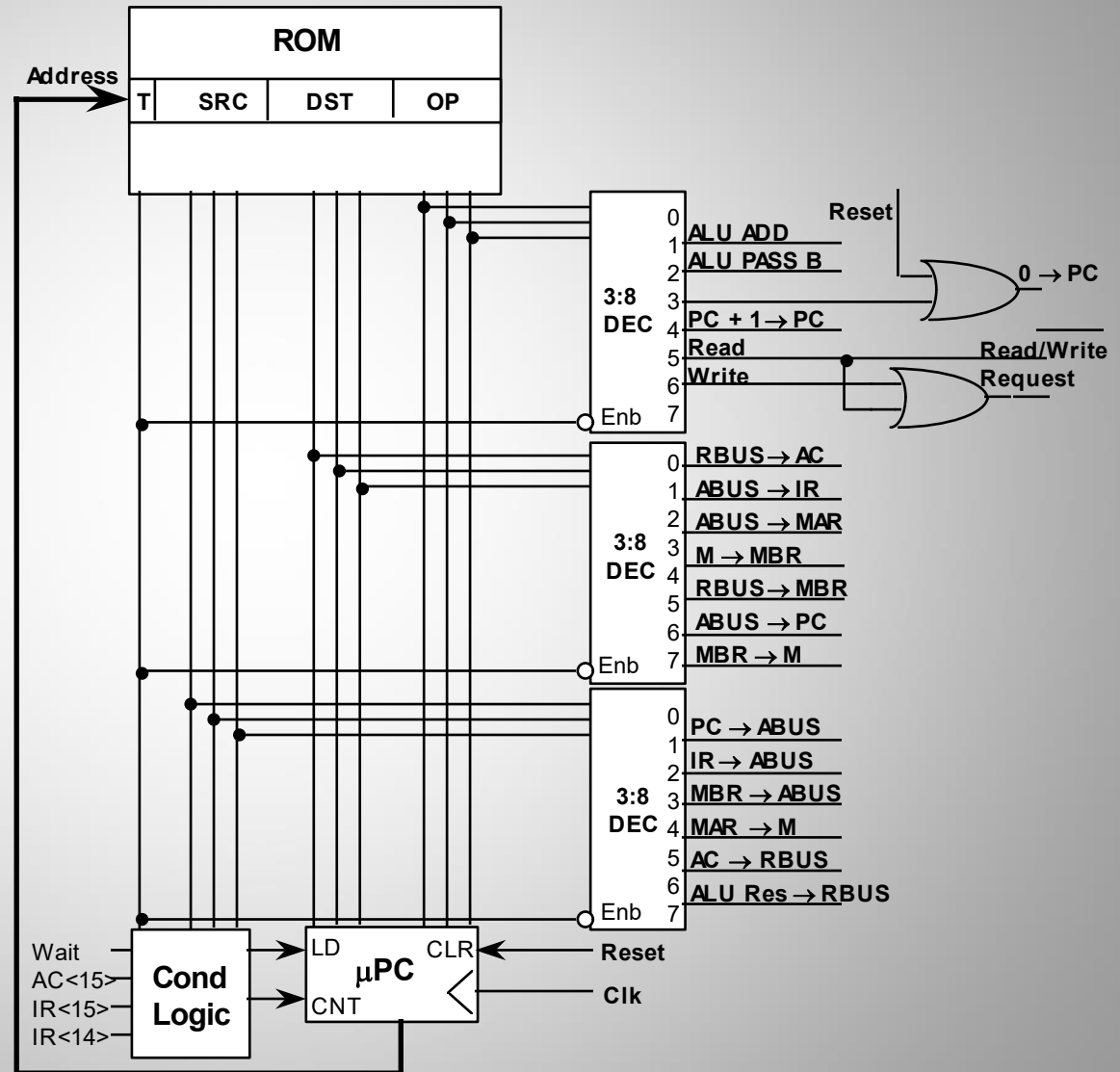
Vertical Microprogramming

ROM ADDRESS	SYMBOLIC CONTENTS	BINARY CONTENTS
010000	ST0 RT AC → MBR	0 101 101 000
010001	RT MAR → M, MBR → M, Write	0 100 111 110
010010	ST1 RT MAR → M, MBR → M, Write	0 100 111 110
010011	BJ Wait=0, RES	1 000 000 000
010100	BJ Wait=1, ST1	1 001 010 010
010101	OD1 BJ IR<14>=1, BR0	1 111 011 101
010110	AD0 RT MAR → M, Read	0 100 000 101
010111	AD1 RT MAR → M, M → MBR, Read	0 100 100 101
011000	BJ Wait=1, AD1	1 001 010 111
011001	AD2 RT AC + MBR → AC	0 110 001 001
011010	BJ Wait=0, RES	1 000 000 000
011011	BJ Wait=1, RES	1 000 000 000
011100	BR0 BJ AC<15>=0, RES	1 010 000 000
011101	RT IR → PC	0 010 110 000
011110	BJ AC<15>=1, RES	1 011 000 000

31 words x 10 ROM bits = 310 bits total *versus* 16 x 38 = 608 bits horizontal

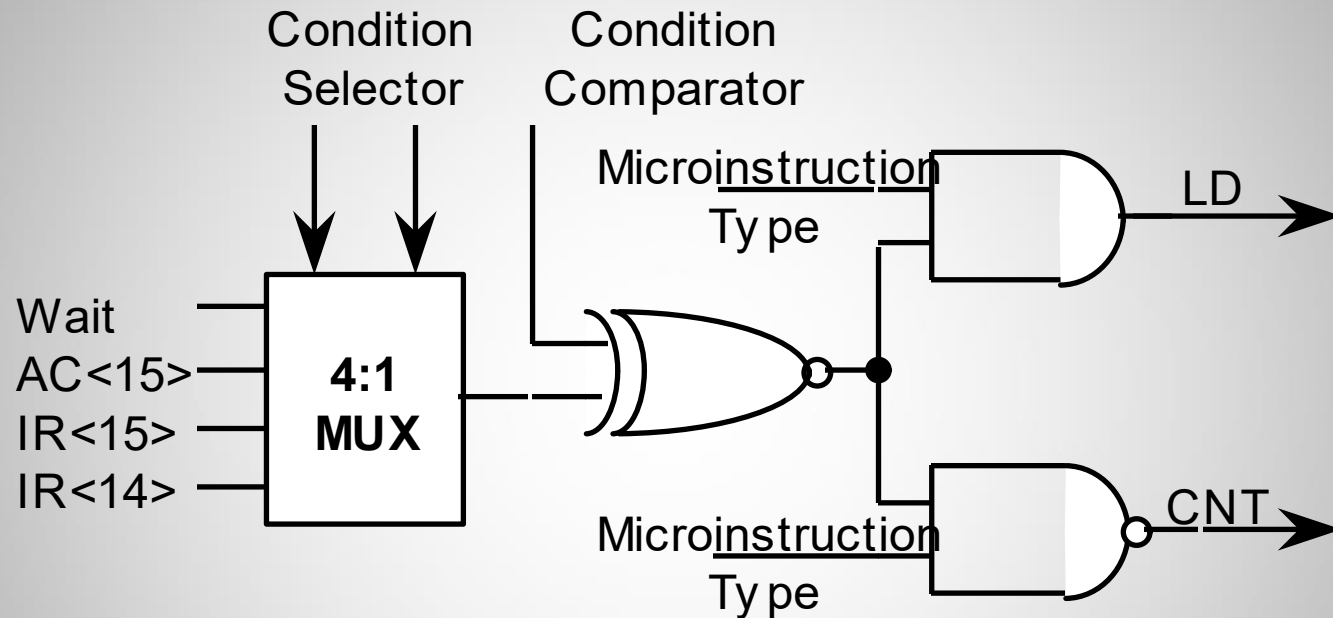
Vertical Programming

Controller Block Diagram



Vertical Microprogramming

Condition Logic



Vertical Microprogramming

- **Writeable Control Store**

- Part of control store addresses map into RAM
 - » Allows assembly language programmer to implement own instructions
 - » Extend "native" instruction set with application specific instructions
 - » Requires considerable sophistication to write microcode
 - » Not a popular approach with today's processors
- Make the native instruction set simple and fast
- Write "higher level" functions as assembly language sequences

课后作业

- 1) 查阅:
 - 国际电路公司的
 - » 8051, PIC, AVR单片机, ARM, MIPS, DSP等微控制器芯片的型号、类型、速度.....upd78F9202, STM8s003, PIC16F886, Mage48
 - 图书馆资源: 电子器件天地, 软件
 - ftp://10.13.71.58
- 2) 习题: /补充讲义P43/学在浙里
- 3) 阅读:
 - 补充讲义 CH.8--9
 - 《数字系统设计与Verilog HDL》书
 - 书CH.11

阅读讲义p.37-51

课后作业

• 4) Project

- **Project1 (必做)**: 将讲义P31页图8.33的框图用具体型号的数字集成电路设计实现为具体电路图。(交设计报告和电路图)
- **Project2 (选做平时成绩加分)**: 用具体集成电路设计并硬件实现一个4bit简单CPU实物。(期末停课前完成, 要有设计报告)

讲义第八章 微控制器设计

-----流水结构与系统架构

8.5 算法与流水结构

- 数字系统设计
 - 确定怎样执行数据处理
 - 提供相应的控制信号
 - 并用数字功能部件构造成电路
- “算法”
 - 指一个问题的求解过程
 - 通过有限个定义的程序性步骤来说明
- 算法结构
 - 算法由许多子运算组成
 - 各子运算的执行方法
 - 子运算间的执行次序

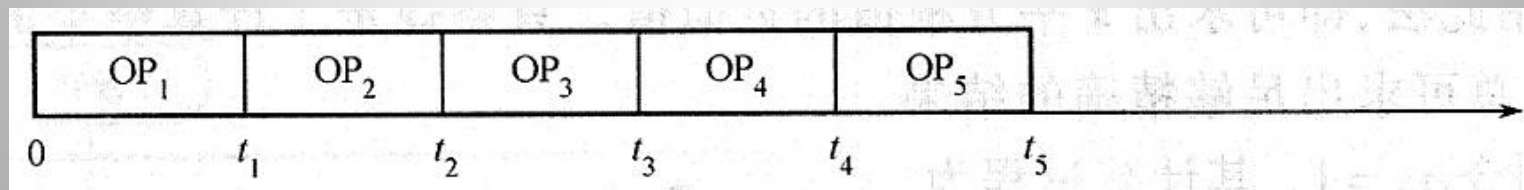
算法结构分类

- 算法结构分类

- 顺序算法结构
- 并行算法结构
- 流水算法结构

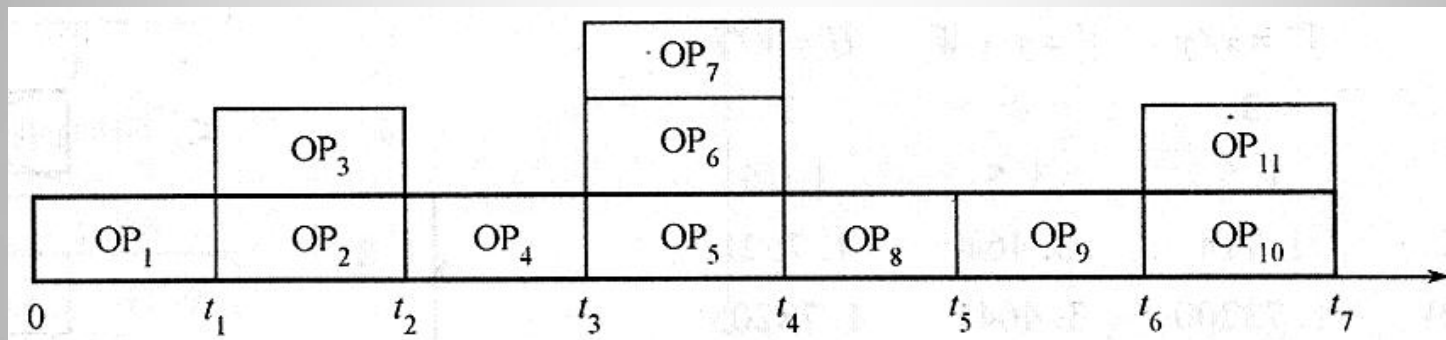
7.5.1 顺序算法结构(串行)

- 顺序算法结构
 - 有-----**2**种
 - 在执行算法的整个过程中
 - 同一时间只进行一种或一组相关的子运算
- 1) 每时间段中仅有一个子运算操作
 - 各子运算之间逐个按规定的次序进行



顺序算法结构

- 2) 在同一时间里，有时有一组子运算操作
 - 同组各操作间相互关联，它们完成后再进行新的一组操作
 - 打包



顺序算法结构

- 顺序算法

- 输入数据D
- 完成算法流程需L个时间段
- 每段的平均时间为 Δt

- 完成运算所需时间

$$\tau = L \cdot \Delta t$$

- 数字系统中

- 待处理的数据是连续输入的数据流
- 数据流中每个元素均完成同样的运算
- 前一数据元素计算完成后，再进行后一数据元素的计算处理
- 若含有n个元素的数据流输入时

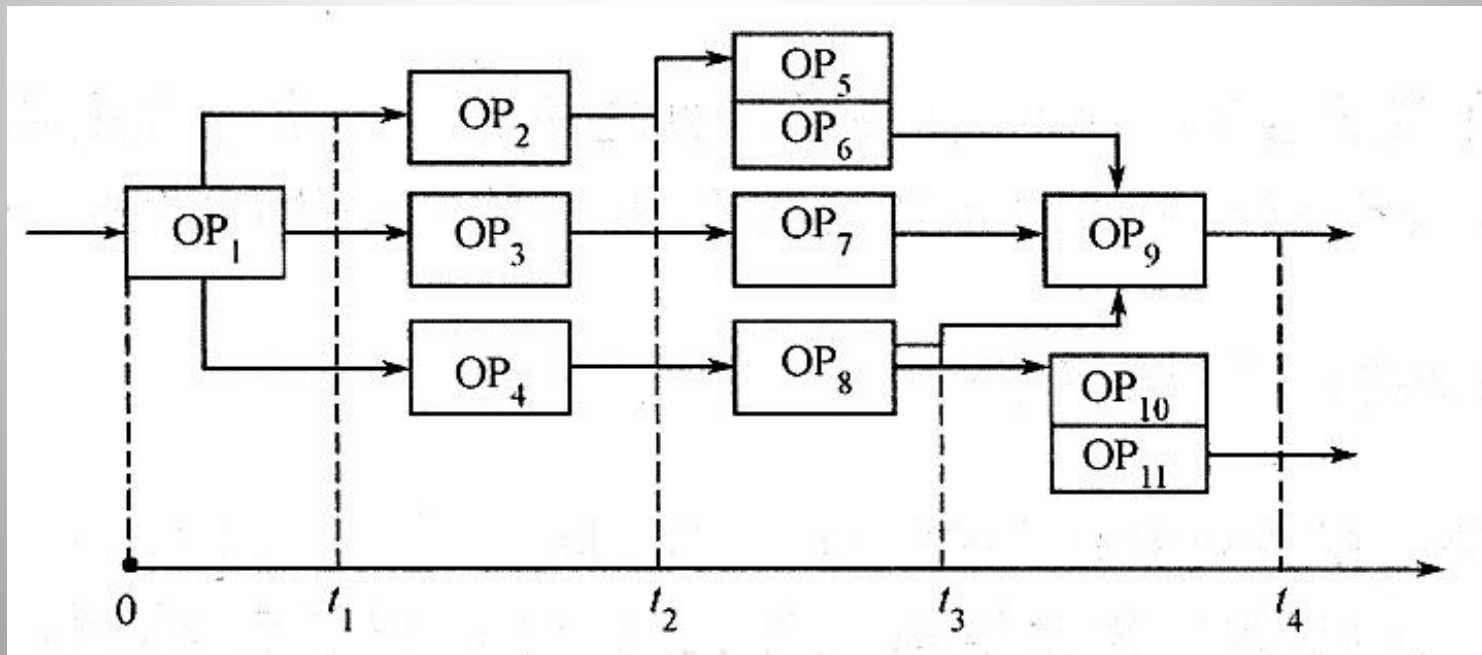
- 总的运算时间为

$$T_s = n \tau = n \cdot L \cdot \Delta t$$

7.5.2 并行算法结构

• 并行算法

- 在同一时间段中，有多条路径在同时进行运算
- 这些同时执行的子运算操作间是相互独立的
 - » 图中**OP2、OP3、OP4**同时进行，三种操作硬件互相独立
 - » **OP5和OP6、OP10和OP11**为顺序运算的一组相关操作



并行算法结构

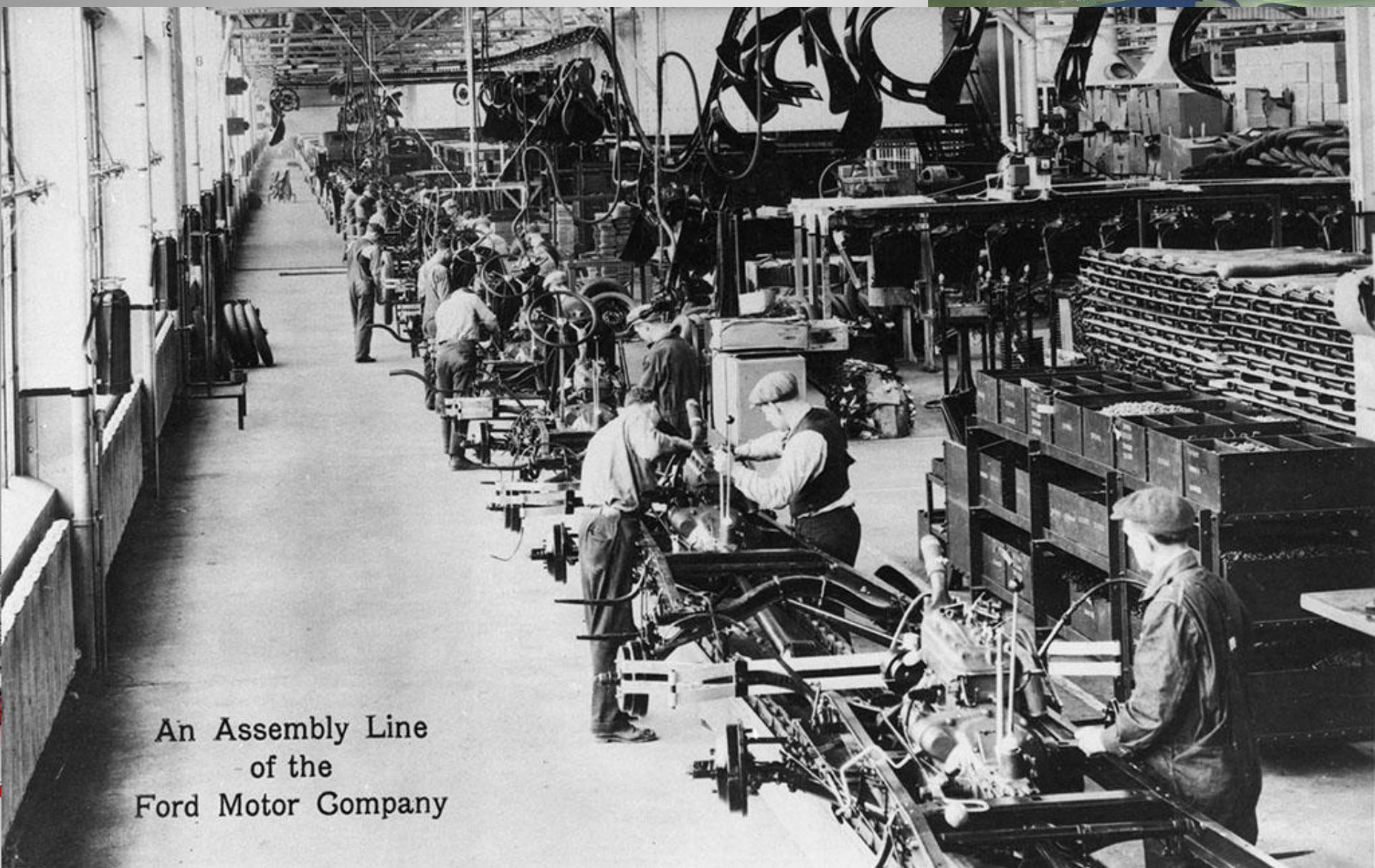
- 并行运算
 - 优点是速度快
 - 缺点是增加硬件结构成本
- 并行算法运算时间
 - 若待处理数据是单元素 D_i (二进制若干位)
 - 完成运算的时间仍满足

$$\tau = L' \cdot \Delta t$$

- 其中 L' 是并行算法流程经过的运算段数
 - L' 比同一系统的顺序算法流程的运算段数 L 要小得多
 - 因此提高了速度
- 数字系统中总运算时间
 - 若含有 n 个元素的数据流输入时
 - 并行结构算法总的运算时间为

$$T_p = n \cdot \tau = n \cdot L' \cdot \Delta t$$

流水线示意图



An Assembly Line
of the
Ford Motor Company

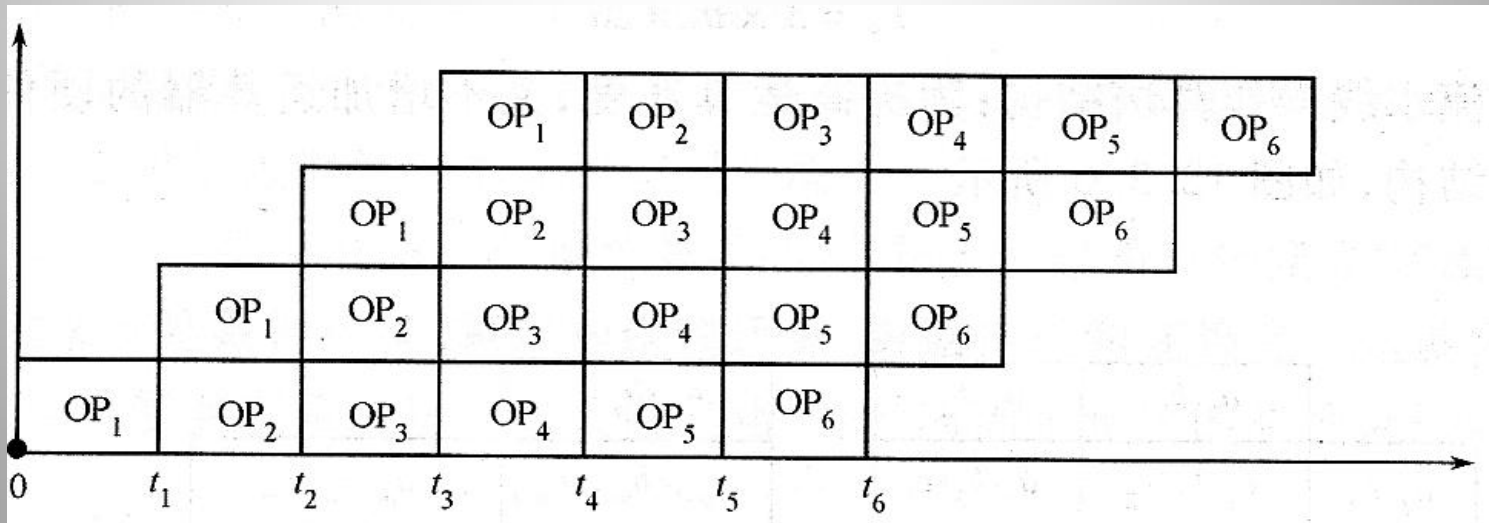
7.5.3 流水线操作算法结构

- 流水线操作算法结构

- 是针对连续输入数据流的系统
- 把整个运算过程分解为若干个段
- 系统在同一时间可对先后输入的数据流元素进行不同段的同时运算

- 条件

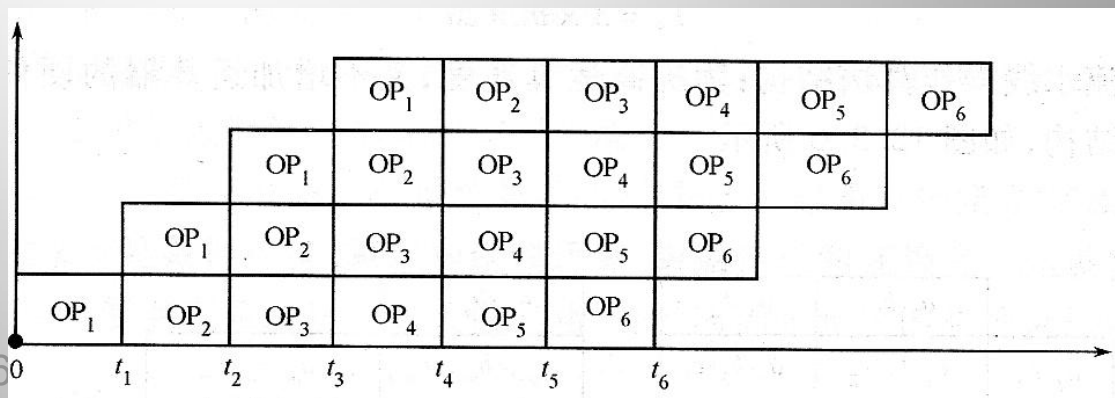
- 有多个操作硬件(不同的)
- 交错: 每一时刻一个硬件不能有2个运算,



流水线结构

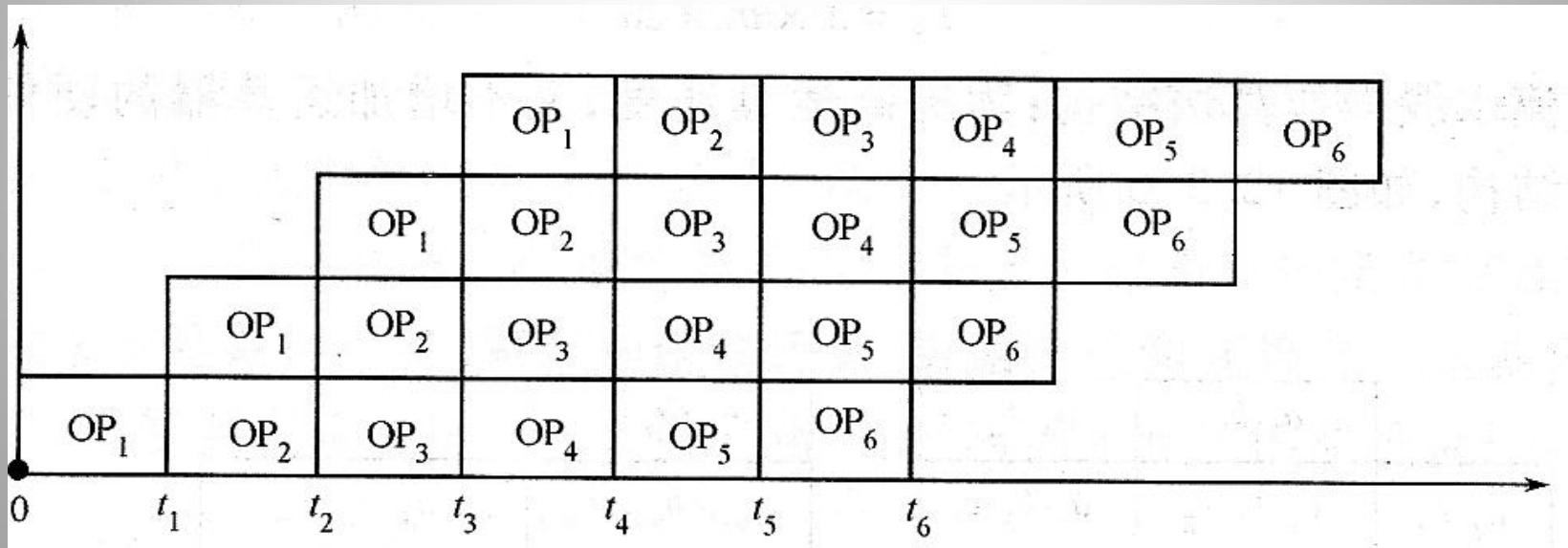
• 特点

- (1) 流水线中必须是连续运算
 - » 只有连续不断地运算才能充分发挥流水线的效率
- (2) 一个运算分解为几个有联系的子运算
 - » 每个子运算由一个专门的功能部件来执行
- (3) 每一个功能部件后都要有一个缓冲寄存器
 - » 用于保存本段的执行结果
- (4) 流水线中各段的时间应尽量相等
 - » 否则将引起“堵塞”或“断流”等现象
- (5) 流水线需要有“装入时间”和“排空时间”
 - » 只有流水线完全充满时，整个流水线的效率才能得到充分发挥



流水线的效率

- 对于有 L 段运算的流水线结构，可以同时处理 L 个数据元素进行不同段的运算，从而大大提高了运算速度。



【例8.14】

- 试用顺序操作算法和流水操作算法实现 $Z = \sqrt{AB + C}$ 运算，并比较运算速度。其中 **A**、**B**、**C** 均为数据流，长度为 **m**，且均是 **n** 位。

解 根据题意，给定的数据流共有 **m** 个元素

其中 $(a_1, b_1, c_1) (a_2, b_2, c_2) \cdots (a_i, b_i, c_i) \cdots (a_m, b_m, c_m)$

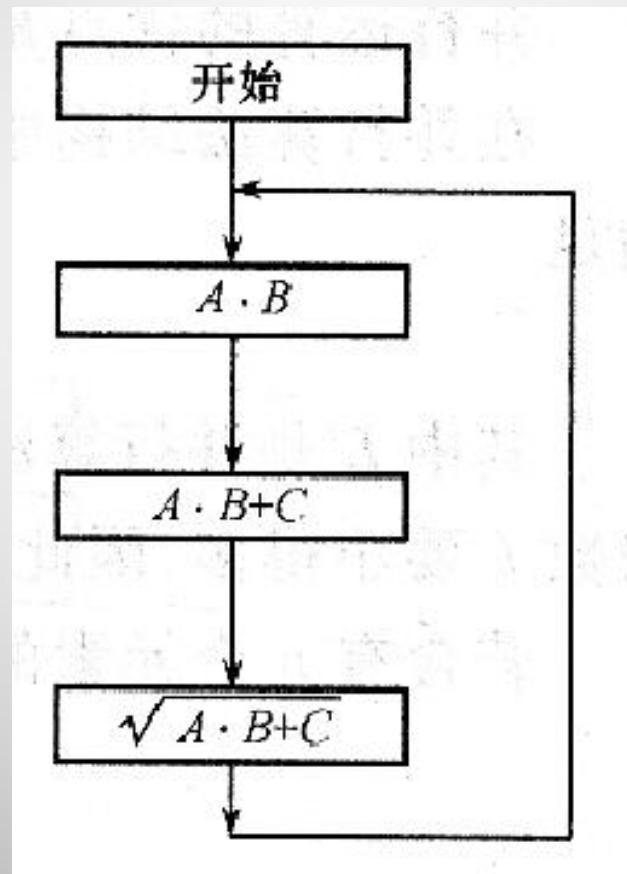
$$a_i = a_i a_i^2 \cdots a_i^n$$

$$b_i = b_i b_i^2 \cdots b_i^n$$

$$c_i = c_i c_i^2 \cdots c_i^n$$

【例8.14】

- 算法流程图
 - 系统运算分解
 - “相乘”、“相加”、“开平方”三个运算段

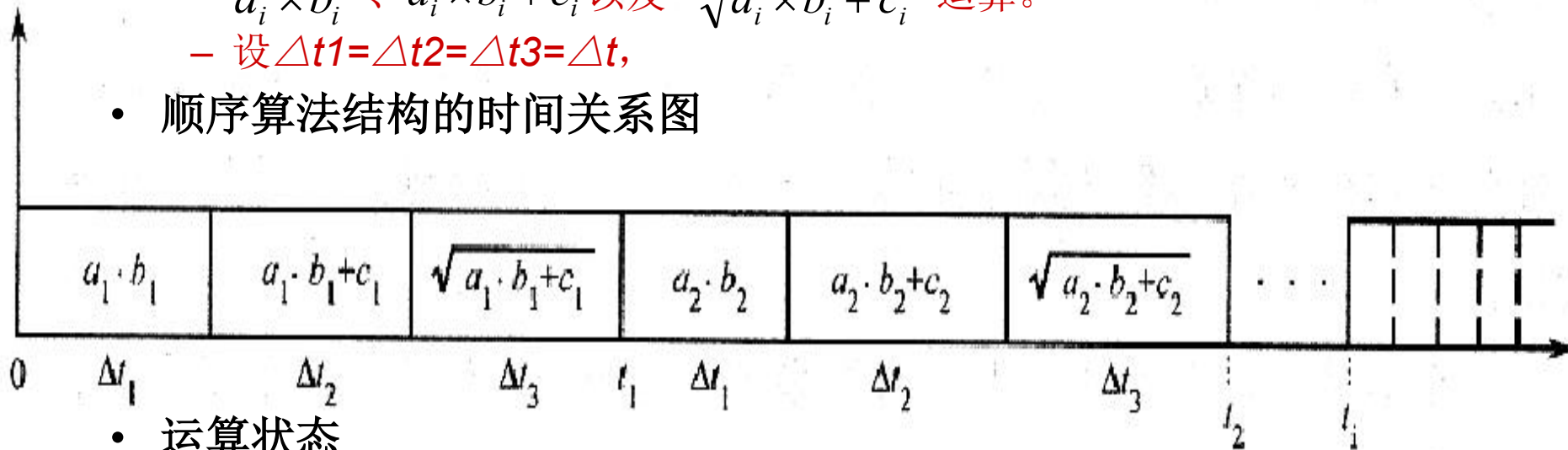


【例8.14】

- (1)顺序算法结构

- 在 Δt_1 、 Δt_2 和 Δt_3 的三段时空中分别完成
- $a_i \times b_i$ 、 $a_i \times b_i + c_i$ 以及 $\sqrt{a_i \times b_i + c_i}$ 运算。
- 设 $\Delta t_1 = \Delta t_2 = \Delta t_3 = \Delta t$,

- 顺序算法结构的时间关系图



- 运算状态

- 第 i 个数据元素在完成前两步运算之后，只有求平方根电路在工作，而乘法和加法电路均处在**闲置**的等待状态，
- 待求平方根运算完成后再接受数据流中的 $(i+1)$ 个数据元素

- 完成整个运算的时间为

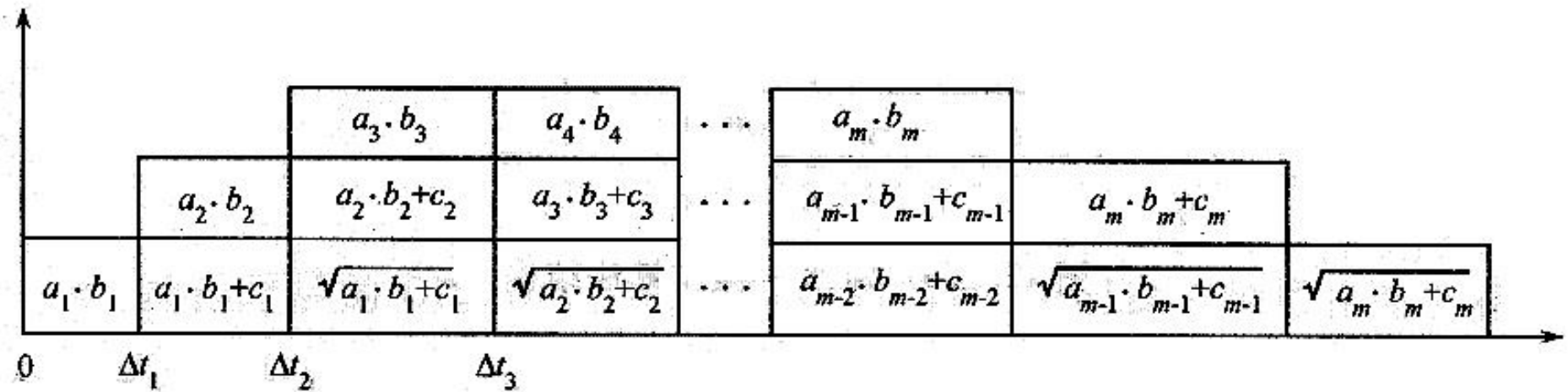
$$T_S = 3 \times m \times \Delta t$$

$$T_S = m \cdot L \cdot \Delta t$$

【例8.14】

• (2)流水算法结构

- 提高运算速度,
- 又不增加运算器的硬件成本
- 改用流水线操作算法结构



- 完成全部数据计算所需要的时间为

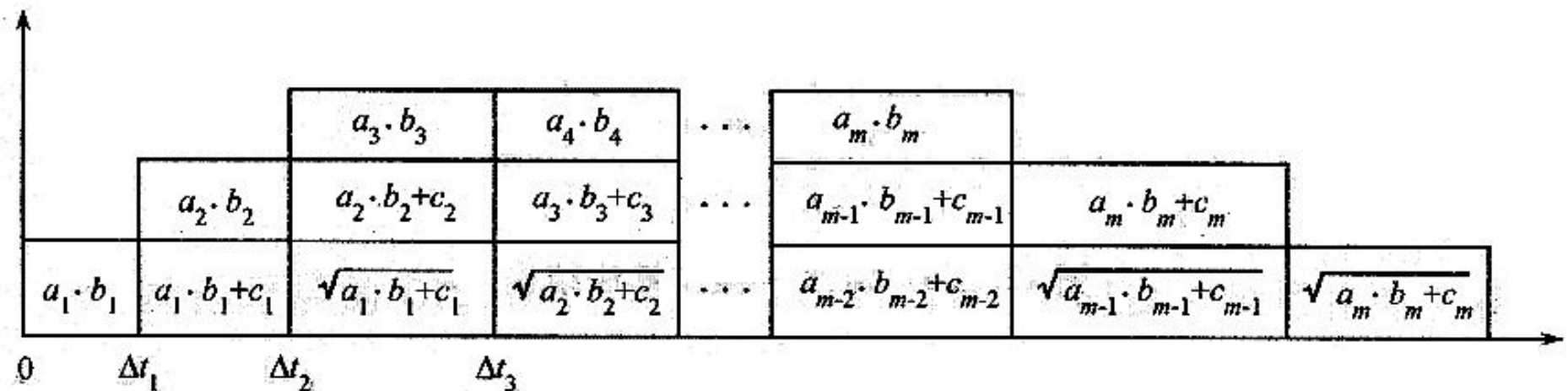
$$T = 3 \times \Delta t + (m - 1) \times \Delta t$$

【例8.14】

• (3) 公式推广

- 系统输入数据流的待处理数据元素为 m 个
- 每个元素运算共计 L 段
- 每段用时间 Δt
- 则流水线操作结构共需要的**运算时间**为

$$T = L \times \Delta t + (m - 1) \times \Delta t$$



小结

- 流水特点
 - 随着运算 L 和 m 的增加
 - 流水线操作时间急剧减少
 - 控制运算稍微复杂一些

【例8.15】

- 某系统待处理数据元素为100个，每个元素需进行16段运算，且每段需运算时间为0.2 μ s，试求顺序结构和流水线操作结构所需要的运算时间。

解 (1)顺序算法结构共需时间:

$$T_s = 100 \times 16 \times 0.2 = 320(\mu s)。$$

(2)采用流水线操作结构时，则需要的时间为:

$$T = 16 \times 0.2 + (100 - 1) \times 0.2 = 23(\mu s)$$

8.6 系统架构、设计与实现

- 系统实现阶段
 - 和设计阶段比起来
 - » 重要性和难度都有下降
 - 主要是工作量方面的问题
 - 实现过程中可能出现的新问题
 - » 在设计阶段没有考虑到或错误估计的
 - » 需要根据情况采取补救措施
 - » 甚至需要进行重新设计

系统架构

- 在设计方案中考虑不同的实现技术
 - 实现所采取的方法肯定也不相同
- 系统架构
 - 系统基于现有芯片的设计
 - 设计方案中涉及到通用处理器或者DSP
 - 设计方案中采用了可编程器件
 - 设计方案中采用ASIC路线

基于现有芯片架构的设计

- 系统基于现有通用芯片的设计
 - 设计原理图
 - 实现阶段的主要任务
 - 设计PCB板
 - » 主要考虑电子器件如何布局
 - » 如何走线
 - » 如何保证速度
 - » 如何控制面积
 - » 以及如何满足电磁兼容性等专业问题。
- 适合系统
 - 数据处理简单
 - 控制不复杂
 - 系统规模小，一般为廉价系统
 - 批量不限

基于处理器架构的设计

- 设计方案中涉及到通用处理器或者**DSP**
 - 实现阶段的任务
 - 接口电路设计
 - **PCB**设计
 - 根据设计好的算法编制程序
- **4位-8位单片机系统**
 - 数据处理简单
 - 控制灵活
 - 可带数码显示，主流系统，价格优势，批量不限
- **32位嵌入式系统**
 - 数据处理复杂
 - 有人机界面要求，可带图形显示
 - 有网络通信要求
 - 系统有一定规模，主流复杂系统，批量不限

基于可编程器件架构的设计

- 设计方案中采用了可编程器件
 - 实现阶段的任务
 - 实现阶段需要把设计好的逻辑进行编译
 - 转换成目标可编程器件的编程控制流
 - 通过专用编程工具进行编程
- 适合系统
 - 控制结构相对固定
 - 数据处理特别或有速度要求
 - 系统有一定特殊性
 - 中小批量

基于ASIC路线的设计

- 设计方案中采用**ASIC**路线
 - 实现阶段的任务
 - 就是把设计交给集成电路生产厂
 - 完成芯片的生产
 - 设计最后包含了多块芯片
 - 也存在PCB设计的问题
- 适合系统
 - 特殊或专用功能
 - 控制结构相对固定
 - 数据处理特别或有速度要求
 - 系统有一定特殊性
 - 大批量，500,000以上

复杂数字系统

- 复杂数字系统的组成
 - 数据处理通道
 - 控制器
 - 时钟
 - 电源
 - 系统监控单元
 - 功耗控制
- 复杂数字系统实现技术
 - 有可能同时需要各种不同的实现技术
 - 可能既需要通用处理器、DSP
 - 又需要现有芯片
 - 同时还需要设计专用芯片
- 复杂数字系统实现
 - 需要多方面的人才进行协作才能很好的实现整个系统
 - 团队合作的重要性

数字系统

- 调试过程

- 不管是PCB板、CPU程序还是设计出来的ASIC
- 都需要进行调试
- 一般先进行独立调试
- 然后再组装成一个完整的系统进行总体联调

- 如果发现新的问题

- 要定位问题出现的原因
 - » 如果是实现阶段的问题，就必须修改实现
 - » 如果是设计或者需求的问题，则可以采取适当的补救措施
 - » 问题严重的需要重新设计。

- 经过调试后，紧接着就是严格的测试

- 根据需求分析的结果形成测试方案
- 对系统的功能和技术参数进行详细的测试
- 保证产品的正确性。

数字系统 之实现

- 测试工作往往是繁琐和复杂的工作
- 特别是测试一些复杂芯片时，没有适当的支持几乎是不可能的，测试成本也在不断增加。
 - 目前在集成电路设计时比较注重可测性设计
 - 就是在设计中就考虑对测试的支持
 - 通过内部状态寄存器、内部测试电路以及扫描链等硬件电路支持边界扫描等测试技术来支持芯片测试，大大提高测试速度。
- 经过测试，确保正确的系统

批量生产

- 批量生产

- 生产阶段的任务是如何在保证质量的前提下提高生产率
 - » 减少废品率
 - » 降低生产成本
- 这些主要涉及到一个生产过程控制的问题
- 存在一整套成熟的生产阶段的理论和经验。

复杂数字系统例

- 课堂讨论: 水泵控制器的功能设计
 - 会有哪些可能的功能?
 - 复杂水泵控制器的数字系统
 - 怎么实现?
 - 画框图
 - 传感器问题

课后作业

- 1) 查阅：
 - 国际电路公司的芯片的型号、类型、速度
 - » 8051, PIC, AVR, NEC, ST单片机,
 - » ARM, MIPS, DSP等微控制器.....
 - 图书馆资源: 电子器件天地, 软件
- 2) 阅读：
 - 复习例题
 - 补充讲义 CH.8
 - 《数字系统设计与Verilog HDL》书
 - 书CH.11

课后作业

- 3) Project

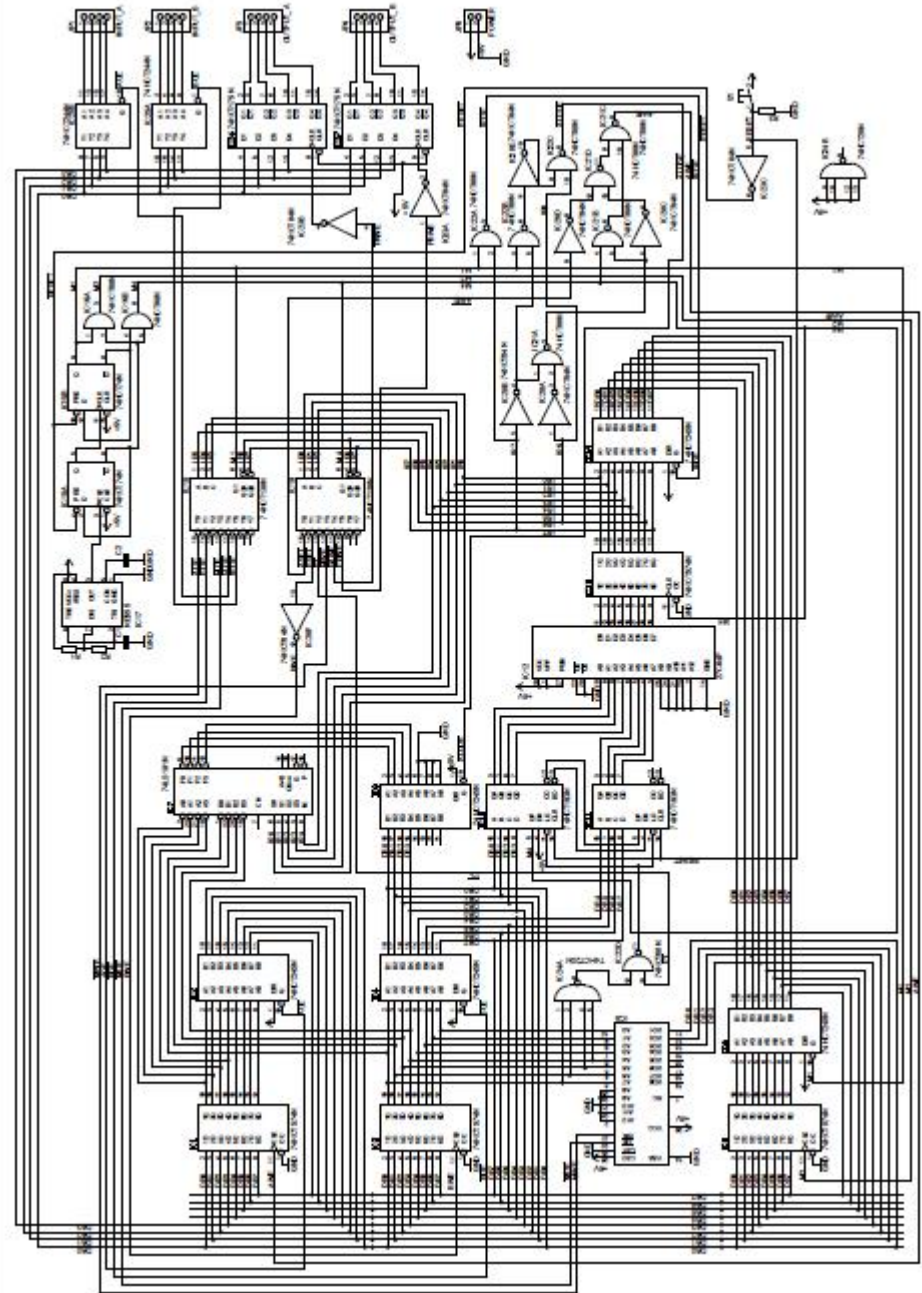
- **Project1（必做）**：将讲义P31页图8.33的框图用具体型号的数字集成电路设计实现为具体电路图。（交设计报告和电路图）
- **Project2（选做平时成绩加分）**：用具体逻辑电路描述设计并FPGA实现一个4bit简单CPU实物。（期末考试停课一周前完成）

- 4) 阅读：

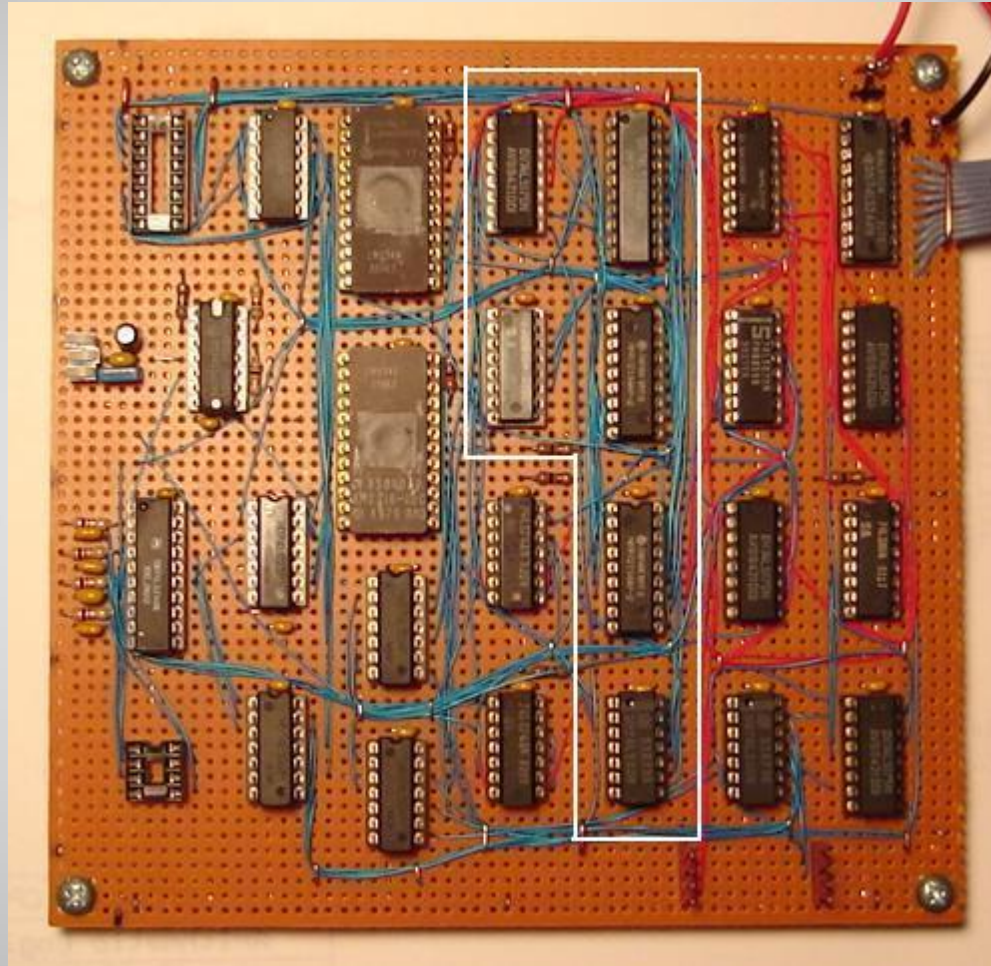
- 补充讲义
- 《数字系统设计与Verilog HDL》书

4bit Simple CPU

- ALU
- 时钟
- PC计数器
- 寄存器
- 程序存储器
- 寄存器组
- 总线



4bit Simple CPU



参考资料

- www.ttlcpu.com